Building data for ARC

Russell Kirk
definitely.not.russell.kirk@gmail.com

November 2026

Abstract

A synthetic ARC task needs a correct label and demonstrations that determine it. Executable rules make correct labels cheap; the work is in the lesson, the choice of which demonstrations a learner sees together. We describe ARCpuzzles, 4,061 procedural generators for ARC-style tasks, each a Python input sampler paired with a Racket reference rule that computes every label. 1,984 are approved for training, each approval bound to the exact reviewed source. We release all of them, unfinished ones included, with labels that say how far each got. To make lessons determine their rule, 1,352 approved generators declare rival rules, 1,007 schedule inputs that refute them, and 941 parametric families add 1,233 rule variants whose lessons must identify the variant that produced them. Agreement between label programs, the check used by RE-ARC and NVARC, does not make a lesson identifiable: a blind solve finds an NVARC puzzle whose first four demonstrations fit a second rule that is wrong on 17 of its 30 pairs, and a screen asks for more demonstrations on 2–4% of RE-ARC, ARC-GEN and NVARC episodes and on 1.8% of ours, which were not screened before training. This is science on a budget, and we report the ideas and pitfalls it produced. A Qwen3.8-27B trained on these data for 273 GPU-hours scored 5.00 on the public leaderboard; it and its per-generator mastery, measured by prefill scans, are described in the appendix. The release is at https://github.com/definitelynotrussellkirk-bit/building-data-for-arc.

1 Introduction

An ARC-AGI-2 puzzle is implicitly a function f from grids to grids. A solver sees a few input/output pairs, the demonstrations, and has to infer f from them to answer a test input, the query. We call the demonstrations together with the query input a *lesson*. ARCpuzzles works in the opposite direction: it starts from f and produces the pairs. For each puzzle it writes f explicitly as a small Racket program. It then asks a second question: which input grids show the rule at work, and which hide it, for example an input on which the rule changes nothing? Each generator answers that question in the Python code that draws its inputs. Outputs are never written by hand; they are computed by running f on the generated input.

NVARC and the 2026 third-place ARC-AGI-2 solution train models on synthetic puzzles at scale [7, 8]. Their checks ask whether several independently written programs agree on each output. Agreement is evidence that an output follows the author's intended rule. It is not evidence that the demonstrations shown to the learner determine that rule: if a simpler rival rule also fits every demonstration but gives a different answer on the query, the lesson is ambiguous however consistent its labels are.

No finite set of demonstrations determines a rule uniquely. For demonstrations $D = \{(x_i, f(x_i))\}$ and any query x_q outside them, the function f' that equals f everywhere except $f'(x_q) \neq f(x_q)$ fits all of D and changes the answer. Ruling out alternatives is also expensive: each rival has to be written and executed against every lesson. The achievable aim is a *sensible* lesson, one whose

demonstrations refute the rules a solver would plausibly infer instead of the intended one. So whenever this paper says a lesson determines its rule, it means among a finite list of rival rules written down for that generator.

Most checks in this paper are therefore not needed for correctness. Every pair a generator emits is true of its rule, because the rule computed it. The checks act on the data the way a regularizer acts on a model: they bias each generator toward lessons in which its rule is tractable to infer and, among plausible rivals, unique. A lesson that fails one of them is still correct; it is a worse lesson.

Most of the work described here therefore did not make the generators more correct. A rule is correct, in the only sense we can check, once it reproduces its original pairs and runs as reviewed; a few stages guard exactly that. The rest of the work made better lessons: it changed which inputs a generator draws and which of them appear together, so that the demonstrations show the differences a learner might otherwise miss. Rival rules, lesson schedules, textures, parametric certificates, review and blind solving are the tools for that (Sections 5–13). We quote the instructions reviewers worked from (Appendix F) because they say concretely what counted as a rule worth keeping and a lesson worth showing: a parameter must change the transformation itself, not only grid sizes, positions or distractors; each claim about what a lesson teaches names an actual lesson, a concrete simpler rival and the observation that tells them apart; where a rule has an intermediate step, changing its result must change what the rule does next; and a one-step rule is legitimate, so operations are not added just to qualify for a category such as multi-rule interaction.

An ideal lesson is lossless but hard to invert. Lossless means its demonstrations fix the query answer among sensible rules; hard to invert means finding that rule still takes reasoning. Compare recovering a pair of primes (p, q) from $p \cdot q$ with recovering it from $p + q$. The product determines the pair up to order, by unique factorization, yet finding the pair is hard. The sum loses it: $3 + 7 = 5 + 5$. Demonstrations drawn without care behave like the sum, because several sensible rules produce the same pairs and the lesson cannot be inverted. Lessons aim at the product. Write $\mathcal{R}$ for a class of sensible rules and $L(f)$ for the demonstrations a lesson shows under rule f; the lesson is *lossless on* $\mathcal{R}$ when

$$f, g \in \mathcal{R}, \quad L(f) = L(g) \;\Rightarrow\; f(x_q) = g(x_q).$$

Recovering f may still take reasoning, as factoring a product does. `bee63e6e52bd` (Section 2) shows both cases: on its original pairs the intended rule and "assume both arms have length 2" give identical outputs, like a sum, and the generator's scheduled arms of lengths 1 and 3 make the lesson determine the rule, like a product.

An initial version of this library was released publicly on 10 May 2026 [1], with a static generator linter as its only automatic check; the checks, rival rules, parametric families, reviewer packets and source-bound approval described here were added since.

ARCpuzzles treats correct labels and sensible lessons as requirements. The point of this paper is producing better lessons, and making the data understandable so that others can use it and continue it: what is in the release, how it was built and checked, how far each check reaches, and how to use it. Unless it says otherwise, the paper describes only the 1,984 approved generators, the ones we trained on and stand behind; the release also holds the 2,077 others. This is science on a budget: one author working with AI agents, and 273 GPU-hours and 379,598 training examples for the submission run, where NVARC's solution trained on 3.2 million augmented samples [7]. Where a controlled comparison would have cost about as much as the change it tested, we made the change and measured afterwards, and many choices traded consistency for efficiency (Section 14, Appendix D). The release reflects the same budget: rather than polish a small set, such as the hundred best generators, we release all of them in whatever state they reached, with labels that say which state that is. Section 19 answers the six judging criteria directly.

2 Examples

We start with examples. The first three were picked by hand to show what the work adds; the six after them were drawn at random from the approved generators and are not chosen to flatter.

Picked by hand.

- `57bafcdde8c3` recolours to cyan every non-background component larger than one cell, leaving single cells alone. It is a parametric family: one rule with six settings (minimum component size 2, 3 or 4, counted with 4- or 8-connectivity), all run on the same sampled inputs. Every lesson is checked so that its demonstrations rule out every setting except the one that produced it (Figure 1).
- `581bb11ff645` sorts complete red column groups, delimited by separators, into their original slots by increasing count of black holes. Other settings of the same family count holes as connected regions and sort in either direction. The trained model mastered only 6.7% of fresh draws, meaning it gave every answer token probability at least 1/2 (Figure 2, Appendix D).
- `bee63e6e52bd` pours yellow from the gap in a frame through the cavity and diagonally past both arm tips. A reviewer found that every original pair has arms of length 2, so "assume 2/2 arms" fits them all; the generator now cycles arm lengths 2/2, 1/1, 1/3 and 3/1 by sample number, so three of every four draws refute that shortcut (Figure 3, Section 11). The original pairs lose the rule the way a sum loses its terms; the scheduled lesson keeps it, as a product keeps its factors (Section 1).

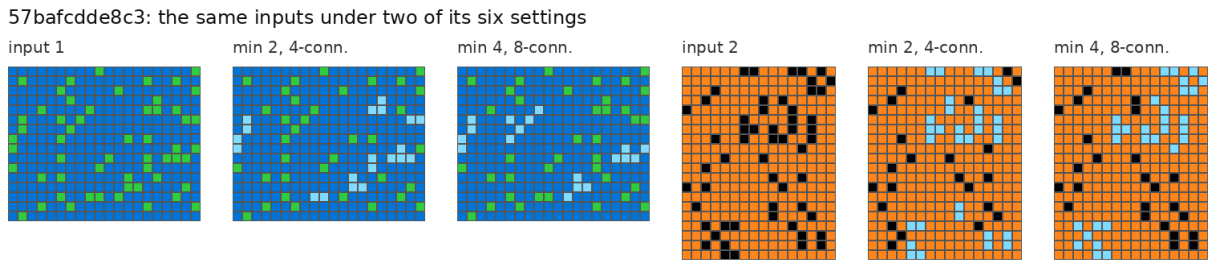


Figure 1: `57bafcdde8c3`: two demonstration inputs and their outputs under two of the family's six settings. Larger minimum size and wider connectivity change which components turn cyan.

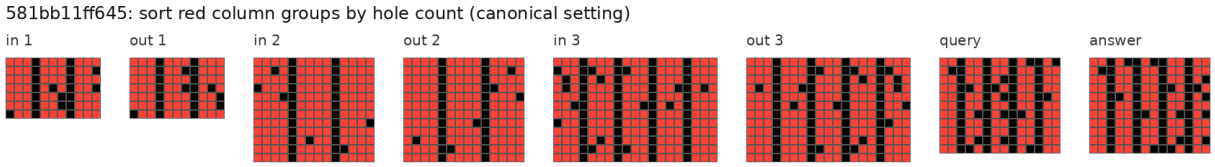


Figure 2: `581bb11ff645`: three demonstrations and a query, canonical setting.

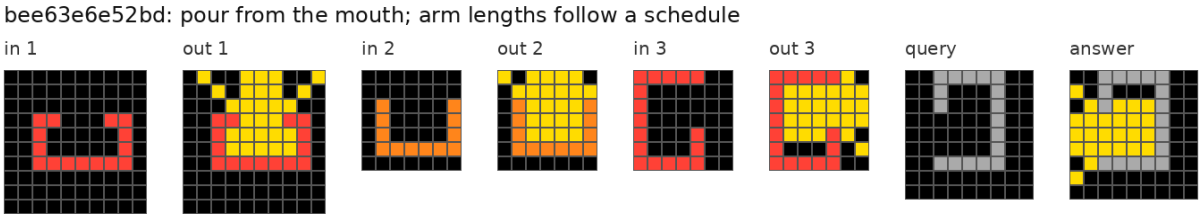


Figure 3: `bee63e6e52bd`: one lesson (seed 17, sample indices 0–3) with arm lengths 2/2, 1/1, 1/3 and 3/1.

Drawn at random. Figure 4 shows two demonstrations from each of six generators drawn uniformly from the 1,984 approved ones (seed 20261108). Section 5.3 measures how many distinct instances each can produce and what lesson structure each has. Appendix D shows the generators the trained model found hardest.

3 How the data was made

Source	Generators
ARC-AGI-2 training tasks	1,000
ARC-AGI-2 public evaluation tasks	120
Puzzle bank (Claude and GPT Pro, author-seeded)	2,859
Custom	30
Untagged (49 generic concept generators, 3 per-puzzle)	52
Total	4,061

Table 1: Generator sources.

The project started on 26 March 2026, the day after ARC Prize 2026 opened. Version 0.1 was released on 10 May, every generator was reviewed between 16 July and 16 August, reviewer packets ran from 3 to 26 October, and the submission model trained from 22 October to 2 November.

Generators move through the following stages; a generator's label says how far it has got, and many have not finished:

1. **Rule.** A reference rule is written in a constrained Racket DSL. It is the only source of output labels.
2. **Generator.** A Python sampler is written against a shared runtime. It states the rule, its sampling axes and its invariants, and optionally declares textures that teach or hide the rule, rival rules and a lesson check.
3. **Battery.** Automatic checks, now twelve, run on the generator and rule together; stages added later are not re-run on generators approved before them.
4. **Review.** A reviewer receives a packet for the generator, attacks it, repairs or extends it, and returns the change with evidence.
5. **Approval.** An accepted review is recorded against the exact generator and rule hashes. Any later edit revokes approval.
6. **Export.** Only approved generators are sampled for training, with answers kept apart from the model-visible challenges.

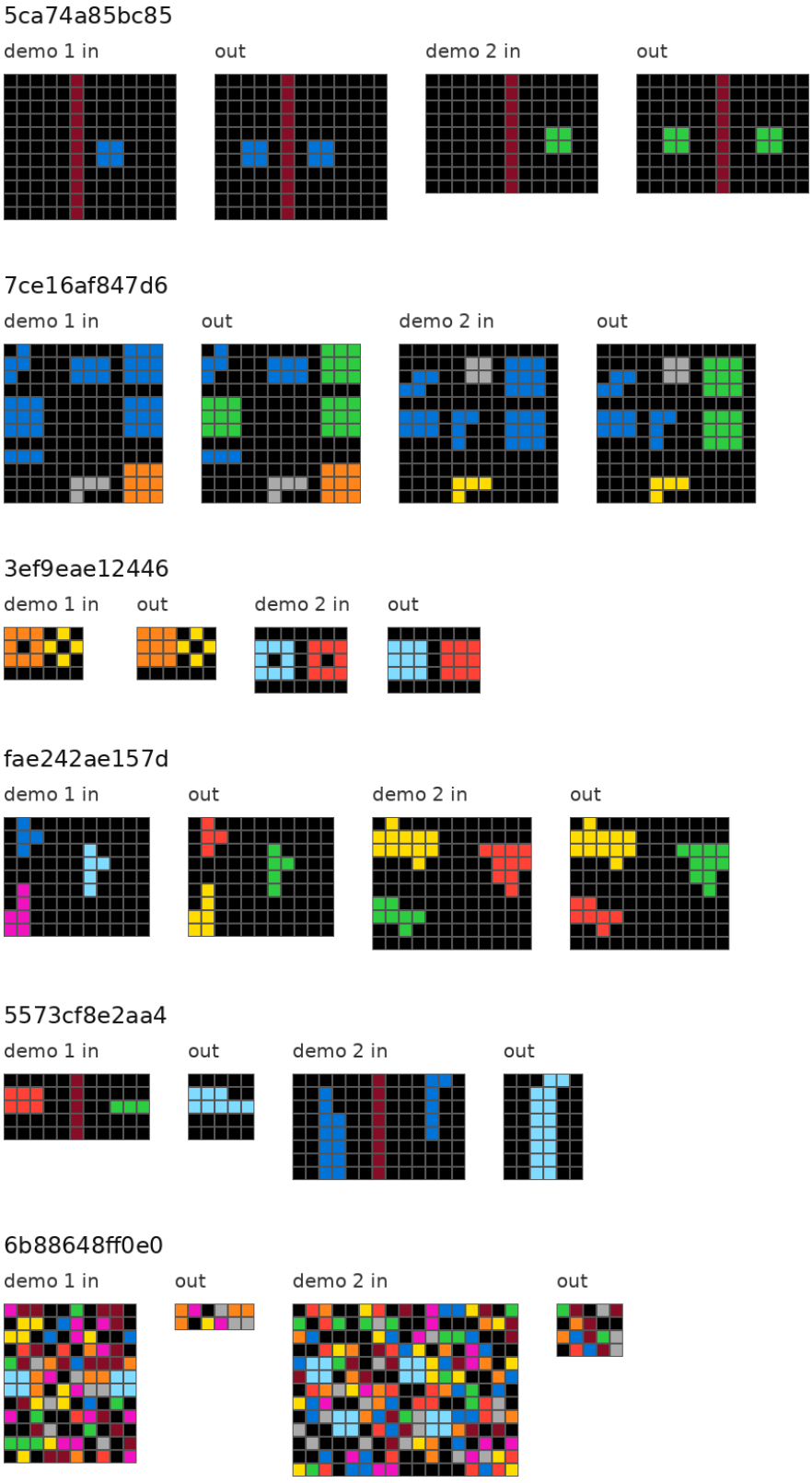


Figure 4: Two demonstrations from each of the six randomly drawn approved generators.

Puzzle banks were written outside the repository by Claude and GPT Pro, the models available at the time, seeded with the author's own puzzle ideas. They came as 21-puzzle bundles (7 easy, 7 medium, 7 hard, each with four training pairs, a test pair, a written solution and a verified Python solver), were imported from 16 April, and their solvers were later converted to Racket. The first generators were written the same way. Later generators were written and repaired by coding agents under the author's direction; the July review records Codex, Kimi and OpenAI models as reviewers, and the October reviews were done by Claude and GPT Pro (Section 11).

4 What a generator does

A generator draws fresh tasks for one ARC rule, as many as its input space allows; for the six measured in Section 5.3 it ranges from 2,376 grids to billions. It has two halves. A Python sampler, `generate(seed, sample_index, difficulty, **overrides)`, returns one input grid. A Racket reference rule computes the output. The sampler never writes outputs.

Most puzzles are identified by a 12-character content hash, a SHA-256 prefix over their train and test pairs; 172 identifiers, 121 of them approved, use other forms such as `public_eval_` names. Each generator lives in a directory named by its identifier. Augmented variants that exercise the same rule share their parent's identifier and generator.

`arcpuzzles generate` draws inputs at successive sample indices, drops any that fail the generator's `self_check`, labels the rest with the Racket rule, and assembles demonstrations and held-out queries into an episode. Each episode records the executed rule hash, the hash of every input, and the result of its lesson check.

Generator `0005ab5b7704` ("Marked Column Compression") keeps the columns whose row-0 header is yellow. Its complete reference rule:

```
(rule!
 ;; Keep only the COLUMNS whose header (row 0) is yellow(4); header dropped.
 (let ((keep (filter (lambda (c) (= (at g 0 c) 4)) (range 0 w))))
   (for/list ((r (in-range 1 h)))
     (for/list ((c (in-list keep))) (at g r c)))))
```

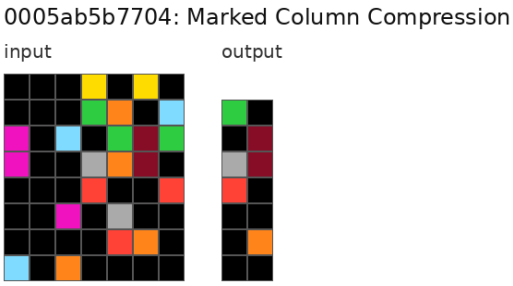


Figure 5: A demonstration generated by `0005ab5b7704` (seed 17). Output rows are drawn level with the input rows they come from.

Why Racket. We chose Racket for three reasons. It hosts a domain-specific language naturally: the prelude's grid primitives (`objects`, `recolor-cells`, `connected-region`, ...) are ordinary definitions, so a rule reads in the vocabulary of the task rather than of a host language. Its prefix notation makes every rule a uniform tree, an operator followed by its arguments, which is simple to

parse, to constrain with a grammar during decoding, and to lint structurally. And it comes with a REPL, so a rule can be built and tested one expression at a time against the demonstrations, by a person or by an agent. The submission did not train a model to write Racket: it answers with the grid directly, and the rules served as the source of every label.

5 Sampling: one shared runtime and keyed draws

All generators share one runtime instead of carrying their own random-number and grid code.

5.1 How an instance is drawn

Fix a generator G with task identifier t, version string v and Racket rule f. Write σ for an episode seed and $k \in \mathbb{N}$ for a sample index.

Named draws. Every random choice inside G is a named draw (`ctx.draw_*` with a label ℓ such as `"grid_h"`), and each label has its own pseudo-random stream, keyed by a hash of (σ, t, k, ℓ, v). A draw's value therefore depends only on those keys and its own arguments: adding, removing or reordering other draws cannot change it. Bare `random` calls are forbidden, so an input grid is a deterministic function $x = G(\sigma, k)$. A caller may override a label within the same bounds.

Episodes. An episode with n demonstrations and m queries tries sample indices $k_0, k_0 + 1, \ldots$ in order. Index k is accepted when $x_k = G(\sigma, k)$ is a well-formed grid, G's `self_check` returns no findings and leaves x_k unchanged, $y_k = f(x_k)$ executes, $y_k \neq x_k$ unless the task allows identity outputs, and (x_k, y_k) is not a repeat. With $k_1 < \cdots < k_{n+m}$ the first $n + m$ accepted indices in $[k_0,\ k_0 + 5(n + m))$, the demonstrations are $(x_{k_i}, f(x_{k_i}))_{i \le n}$ and the queries are the rest. Fewer acceptances fail the episode; every rejected index is logged with its reason.

Exports. An export with seed s over a pool P with mix weights w uses one generator $r = \mathrm{MT}(s)$. Attempt j draws a generator g_j with $\Pr(g_j = g) = w_g / \sum_{h \in P} w_h$ (or uniformly), then an episode seed σ_j of 63 bits, both from r. The same sequence drives planning and execution, including rejected attempts, so an export is reproducible from (s, P, w). The `generate` command uses $\sigma_i = s + i$ for episode i. A parametric export also uses $\sigma_i = s + i$ and draws the rule setting θ_i uniformly from the family's settings Θ with $\mathrm{MT}(s + i)$; every pair of the episode uses f_{θ_i}.

Overrides. Any named axis can be pinned by the caller, and the pinned value passes the same validity check as a random one. The override checker probes every declared axis at its boundaries and in combination, and generators declare which combinations they legitimately refuse.

Axis honesty. Each draw records the bounds it actually used. The defect linter compares those bounds with the declared axes, so a generator cannot claim a range it never samples.

Difficulty. `easy`, `medium` and `hard` select bands within each axis, so a curriculum changes scale without changing the rule.

Shared helpers. Grid, placement, palette, object, shape, panel, room, lattice and texture helpers are shared, so a fix to a placement bug reaches every generator that uses it. The rule runtime also provides 18 rule templates, composable factories for common rule shapes (recolour map, colour swap, recolour by size, fill enclosed regions, gravity, mirror, symmetry completion, upscale, tile, per-object transforms, keep or remove a colour, crop to an object, sort objects, connect dots, extend lines), callable as `template!` in the rule REPL.

Rejections. The runner records every rejected pair with a phase-prefixed reason, and drops duplicate and identity pairs.

5.2 Lessons, not samples

The sample index is not only a source of randomness. In most generators it is a position in a lesson schedule: a function $\pi : \mathbb{N} \to \mathcal{P}$ onto a small set of scene profiles, usually $\pi(k) = \mathcal{P}_{k \bmod p}$, chosen so that every window of consecutive indices contains inputs that refute the rivals of Section 7. The seed then varies the surface (colours, positions, shapes, counts) inside each profile. Because an episode takes accepted indices in order from k_0, a schedule of period p places every profile in any lesson of at least p pairs whose indices are consecutive, that is, when no draw inside the window was rejected. Of the 1,984 approved generators, 1,007 declare such a schedule.

5.3 Reach and expressiveness: six approved generators drawn at random

The set of grids a generator can emit is its image $\mathrm{Im}(G) = \{G(\sigma, k)\}$, and its draw distribution is $p(x) = \Pr[G(\sigma, k) = x]$. We report three quantities.

- S_{obs}: distinct inputs among N draws $(\sigma, k) = (j, j)$, $j < N$; a lower bound on $|\mathrm{Im}(G)|$.
- Chao1, $\hat{S} = S_{\mathrm{obs}} + f_1^2/(2f_2)$, where f_1, f_2 count grids seen once and twice: a lower-bound estimator of $|\mathrm{Im}(G)|$.
- The effective size $N_{\mathrm{eff}} = 1/\sum_x p(x)^2$, the number of equally likely grids with the same collision rate. It satisfies $N_{\mathrm{eff}} \le |\mathrm{Im}(G)|$, with equality for uniform p. Since $2C/(N(N-1))$ is an unbiased estimate of $\sum_x p(x)^2$, where C is the number of colliding pairs among the N draws, we report $\hat{N}_{\mathrm{eff}} = N(N-1)/(2C)$.

We drew six generators uniformly from the 1,984 approved ones (seed 20261108). None was chosen by hand.

An exact case. `3ef9eae12446` (fill the zero centre of a 3×3 ring whose eight border cells share one colour) can be counted directly. Without a difficulty band, $k \bmod 6$ fixes both the grid size $(h, w) \in \{(4,6), (5,7), (6,7), (5,8), (4,7), (6,8)\}$ and the profile. The seed chooses an ordered colour pair ($9 \cdot 8 = 72$ ways), the motif row ($h - 2$ ways) and which end holds the target (2 ways). The `multi_target` phase paints two rings and is symmetric under swapping both colours and ends. Hence

$$|\mathrm{Im}(G)| \;=\; \sum_{\text{phases}} 2 \cdot 72\,(h-2) \;-\; \tfrac{1}{2} \cdot 2 \cdot 72 \cdot 3 \;=\; 2{,}592 - 216 \;=\; 2{,}376,$$

and, with the six phases equally likely and grids uniform within a phase, $N_{\mathrm{eff}} = 2{,}073.6$ exactly. From $N = 20{,}000$ draws we measure $S_{\mathrm{obs}} = 2{,}374$, Chao1 $= 2{,}378$ and $N_{\mathrm{eff}} \approx 2{,}075$, which validates the estimators.

Generator	Declared	$\lvert\Theta\rvert$	N	S_{obs}	Chao1	N_{eff}	Grid shapes
`3ef9eae12446`	196	1	20k	2,374	2,378	2,075	6
`5ca74a85bc85`	16	1	20k	13,349	62,000$^{\uparrow}$	146	24
`5573cf8e2aa4`	150	1	20k	14,837	365,000$^{\uparrow}$	14,000	25
`7ce16af847d6`	3	2	100k	99,997	–	$\approx 1.7 \times 10^9$	2
`fae242ae157d`	54	3	100k	99,998	–	$\approx 2.5 \times 10^9$	6
`6b88648ff0e0`	14	3	100k	100,000	–	$\gtrsim 1.7 \times 10^9$	165

Table 2: Declared configurations ($\prod_j |A_j|$), rule settings, and measured reach. Grid shapes are counted over 5,000 draws. Chao1 values marked $^{\uparrow}$ were still rising with N; they are estimates at the stated N, not bounds. "–": too few repeats for Chao1. For `6b88648ff0e0` no collision occurred; the bound uses $C \leq 3$, the 95% upper limit for zero observed events.

Declared configurations and measured reach disagree in both directions. `7ce16af847d6` declares three configurations and produced 99,997 distinct grids in 100,000 draws; `3ef9eae12446` declares 196 and emits exactly 2,376. The declared axes list the structural choices a generator makes; everything else, such as colours, positions and shapes, varies with the seed.

Generator	Rule	Schedule π	Rivals (fit the original pairs)	Amb.
`3ef9eae12446`	fill centre of a uniform 3×3 ring	k mod 6	1 (1): four-neighbour-only test	0
`7ce16af847d6`	recolour exact solid blue 3×3 components	$(\sigma + k)$ mod 3	5 (3): bbox-only, size-only, contains-window, any colour, all blue	0
`fae242ae157d`	recolour three objects 2/3/4 by area rank	k mod 3, k mod 6	6 (3): scan order, fixed areas 4/5/6, scene position, colour rank, descending, identity	0
`5573cf8e2aa4`	union of two panels' occupancy	anchor at $k \equiv 0 \pmod 3$	5 (1): intersection, XOR, copy left, copy right, cell inequality	0
`5ca74a85bc85`	mirror source cells across a divider	none; 5 replay, 11 general profiles	1 (1): filled source bounding box	4
`6b88648ff0e0`	stack strips between cyan markers	none	0 in production; family audit of 26 policies	–

Table 3: Lesson structure of the six sampled generators. In parentheses: rivals that reproduce every labelled original pair, measured by `arcpuzzles lesson-census`. Amb.: of 60 census lessons (20 per difficulty band, 3 demonstrations and 1 query, explicit demonstration count), those in which a declared rival fits every demonstration but changes the query answer.

What the six show.

- **The original demonstrations are often under-determined.** In five of the six, at least one simple rival fits every labelled original pair. For `5573cf8e2aa4` it is raw cell inequality (mark a cell where the two panels differ): each original pair gives its two panels different

colours, so an occupied overlap is unequal and is marked exactly as union marks it. Ordinary draws of the generator give both panels one shared colour, which separates the two rules. Because these rivals reproduce the test pairs too, they score perfectly on the original task: the ambiguity is invisible to the task's own score, but data that only imitates the original inputs would teach the shortcut as readily as the rule. For `fae242ae157d`, every official example uses areas 4/5/6 in ascending scan order, so "rank by area", "rank by scan order" and "map 4/5/6 to 2/3/4" agree on all of them.

- **Schedules are designed against query rotation.** The six-slot schedule of `3ef9eae12446` places the cornerless-cross witness at three positions so that every cyclic window of four pairs keeps one among its demonstrations, whichever pair is held out as the query. A four-slot schedule was rejected because it could hold out the only witness.
- **Replay anchors trade effective size for coverage of the original.** `5ca74a85bc85` draws its five public profiles with weight 1 and its eleven general profiles with weight 2, so $5/27 \approx 18.5\%$ of draws replay an original input exactly. This concentrates mass ($N_{\text{eff}} = 146$) while the general profiles keep the image large (Chao1 $\geq$ 62,000 and rising).
- **Honest axes replaced advertised ones.** The imported version of `7ce16af847d6` was a fixed five-component fixture that advertised nine controls; the imported `5573cf8e2aa4` declared nine axes, several singleton, aliased or inert. Both rewrites expose only exact, observable axes. Declared reach fell; measured reach rose.
- **Census lessons refute the declared rivals, with one exception.** Four of the five generators that declare rivals produced no ambiguous lesson in 60. `5ca74a85bc85`, which has no schedule, produced 4: the filled-bounding-box rival fits all three demonstrations and changes the query answer. In 21 more its demonstrations do not refute the rival, but the rival gives the right query answer. When export chooses the demonstration count, the worker rejects an ambiguous lesson. When the caller fixes the count, as the census does, the lesson is kept and its provenance records `diagnostic_unsupported`.
- **A declaration can be wrong, and approval does not follow a stricter gate.** The generator `5573cf8e2aa4` sets `CANONICAL_SHORTCUTS` to a copy of `LESSON_COMPETITORS`, declaring all five rivals to fit the original pairs. On 23 October the semantic gate began checking each declared shortcut against the labelled original pairs; intersection, copy-left and copy-right fit none of the three, and XOR fits one. The generator was approved on 3 October, has not changed since, and fails today's gate. Its lessons are not affected (no ambiguous census lesson; every witness fires), but the label "canonical shortcut" is wrong for four of its rivals. 356 approved generators copy their whole rival list into `CANONICAL_SHORTCUTS` this way. Of five drawn from them at random (seed 20261108), two fail today's gate for the same reason (`6d26bc40cbb2`, three shortcuts; `99e7ba062bec`, five) and three pass.
- **The trained model mastered three of the six and found two more easy.** Measured by prefill scans of fresh draws (Appendix D), `3ef9eae12446`, `5573cf8e2aa4` and `5ca74a85bc85` were mastered by the end of the run, `7ce16af847d6` and `6b88648ff0e0` were easy, and `fae242ae157d`, the generator with the most rivals, was medium: 21% of its scanned draws mastered and 25 of 40 greedy answers right.

6 Textures: making the rule visible

A texture is a named kind of input a generator can draw, such as sparse objects, stripes or a full grid. A generator can declare *helpful* textures, which make the rule visible, and *degenerate* textures, which hide it. Degenerate textures are reachable only by explicit override, never on the default

path, and the self-check stage fails a generator if one leaks into ordinary output; the exception is a degenerate texture the generator declares a lawful edge case (`LAWFUL_DEGENERATE_TEXTURES`), which may appear in ordinary draws. Declaring textures is optional: of the 1,984 approved generators, 264 name helpful textures and 255 name degenerate ones (459 distinct names). Counting texture sampling axes and lawful degenerate textures as well, 347 declare textures in some form. Across the whole library, 1,881 generators name degenerate textures.

In `0005ab5b7704`, the invariants require content in both kept and dropped columns, so keep-versus-drop shows in every output. Its degenerate textures are `no_markers`, `all_columns_marked` and `no_body`.

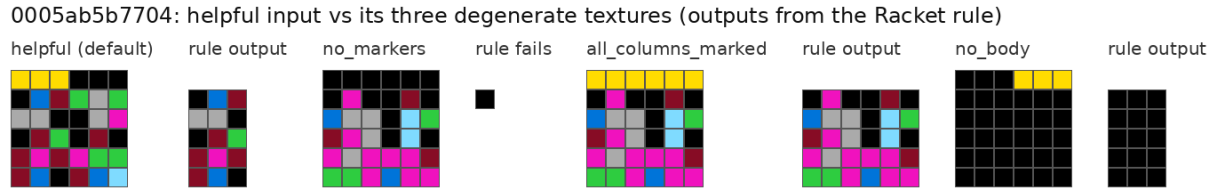


Figure 6: A helpful input of `0005ab5b7704` and its three degenerate textures, labelled by the Racket rule. With no markers the rule has no column to keep and fails (shown as a single black cell); with every column marked the output is the whole body, so keep-versus-drop is invisible; with no body every output cell is black. The generator's `self_check` rejects all three: "row 0 has no color-4 markers", "every column is marked", and "no selected column has a nonzero body cell".

7 Rival rules: attacking the rule

A generator can declare rival rules: plausible alternatives a solver might infer instead of the real one. The semantic gate checks that generated lessons eliminate them.

- `SEMANTIC_RIVALS`: named alternative rules, each a function from grid to grid.
- `SEMANTIC_WITNESSES`, `WITNESS_RIVALS`: input features, each tied to the rivals it must refute. Whenever an input has a witness feature, at least one of the rivals tied to it must give a different output from the reference rule on that input.
- `lesson_identifiable`: rejects a lesson in which a rival fits every demonstration but changes the query answer, when export chooses the demonstration count; with a fixed count the lesson is kept and flagged.
- `CANONICAL_SHORTCUTS`: rivals that also fit the original ARC task's own pairs. Each needs generated counterexamples. Since 23 October the gate also verifies the fit and fails a declared shortcut that misses any labelled original pair; generators approved earlier were not re-gated (Section 5.3).

1,498 generators declare 6,822 rivals, a median of 4 and up to 25 per generator, under 5,085 distinct names. Among the 1,984 approved generators, 1,352 declare rivals, 1,224 a lesson-identifiability check and at least 761 canonical shortcuts (1,223 by a looser text match). Most rivals are written for one task. The most common are `identity` (175 generators), `horizontal_only` (42), `vertical_only` (38), `solid_bbox` (19), `largest_component_only` (15) and `ignore_blockers` (14).

Generator `00ed2544a15e` fills the cells between two red cells with cyan when the segment between them holds nothing else. Two of its rivals reproduce every pair in the original ARC task:

```
def _rival_ignore_blockers(grid):
    """Connect every exactly-two-red aligned span, ignoring other colors.
```

```
    Every interior cell is painted, including a blocker. This reproduces every
    canonical example because none places a distractor inside a red-red span,
    but it disagrees on every generated blocked pair.
    """
```

`simple_pairs_only`, which connects only isolated pairs, also fits all original demonstrations because they contain only disjoint pairs. The original task does not separate these rules from the intended one. The generator does: every ordinary example contains a horizontal and a vertical connection, a pair blocked by another colour, an isolated red, and a chain or shared endpoint (Figure 7).

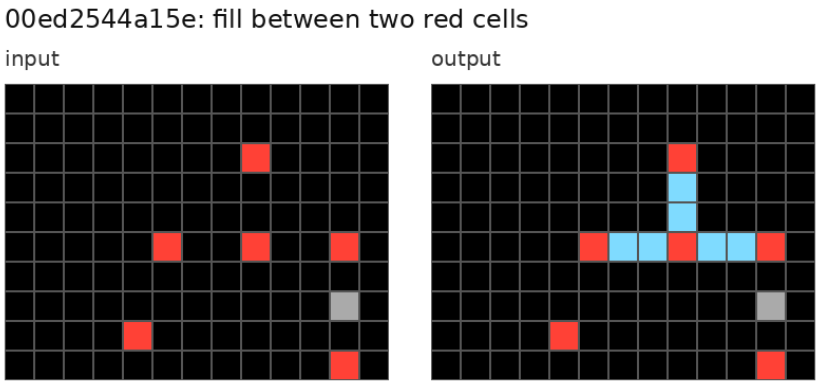


Figure 7: A demonstration generated by `00ed2544a15e` (seed 17): a chain of three reds with a shared endpoint, a vertical connection, a pair blocked by grey (left unfilled), and an isolated red.

Rival rule	Result on the seed-17 lesson
identity	refuted
horizontal pairs only	refuted
vertical pairs only	refuted
ignore blocking colours (fits the original task)	refuted
simple pairs only (fits the original task)	refuted

Table 4: Lesson check recorded in a generated `00ed2544a15e` episode with three demonstrations. A full semantic-gate run on this generator passed on 8 November 2026.

These checks are sampled evidence against named alternatives, not proof that no other rule fits the demonstrations.

8 Templates: parametric Racket rules

941 generators have an accepted parametric family: one input sampler shared by several rule settings, each with its own Racket program. All pairs in an episode use one setting, and the demonstrations must tell the settings apart. The family's declared shortcuts are checked when the family is accepted. Every episode is certified when it is drawn, against the family's settings: each setting's program

runs in Racket on the demonstrations, the drawn setting must be the only one that reproduces them all, and every query label must agree among the survivors; otherwise the episode is rejected. The 941 families have 2,174 settings, each with its own Racket program (2,170 distinct program hashes). In every family the default setting is byte-for-byte the canonical program, so the families add 1,233 rule variants to the library. Most families have one parameter with two settings (759); 29 have two or three parameters, with up to 12 settings.

`0005ab5b7704` adds a setting that orders the selected columns by how many non-black body cells they hold:

```
(rule!
 ;; Stage A selects only header-marked columns and measures their body occupancy.
 (let ((keep (filter (lambda (c) (= (at g 0 c) 4)) (range 0 w))))
   (define (body-count c)
     (length (filter (lambda (r) (not (= (at g r c) 0))) (range 1 h))))
   ;; Stage B orders that selected-and-measured intermediate, then packs the body.
   (let ((ordered (sort keep (lambda (a b)
                               (let ((ca (body-count a)) (cb (body-count b)))
                                 (or (> ca cb) (and (= ca cb) (< a b)))))))))
     (for/list ((r (in-range 1 h)))
       (for/list ((c (in-list ordered))) (at g r c))))))
```

The family keeps an adversarial rule, `wrong_intermediate.rkt`, that counts the marked columns and then keeps that many of the densest columns in the whole grid, marked or not. To refute it, the sampler gives the marked columns distinct densities, forces an order that differs from left to right, and always includes a dense unmarked column. A 32-seed audit and a 256-seed sweep found no surviving declared shortcut. In the exported episode of Figure 8, unmarked column 1 holds seven body cells and outranks the marked columns under the adversarial rule; all three demonstrations of that episode refute it.

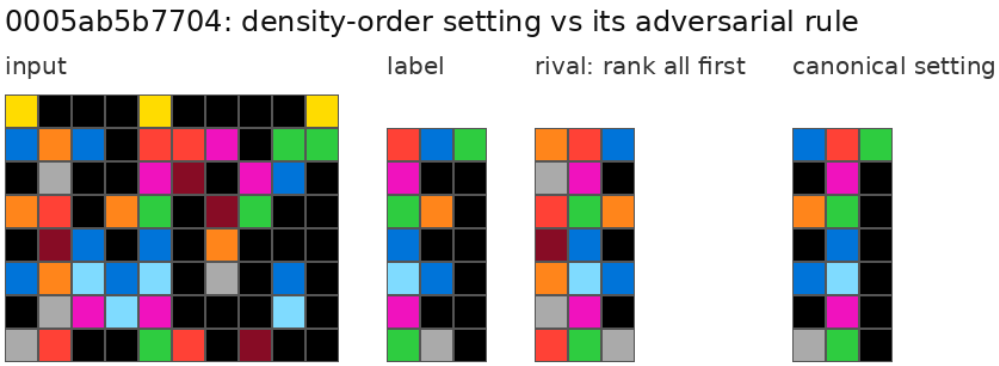


Figure 8: One demonstration from an exported `body_density_desc` episode (seed 12). From left: input; Racket label; the adversarial rule's answer; the canonical setting's answer.

Family	Settings	Lesson requirement
57bafcdde8c3	minimum size 2, 3, 4 × connectivity 4, 8	Separate all six settings and five shortcuts; queries up to 30×30
7b0cb3b9a618	front or back crossing band	Reject eight declared shortcuts under both settings
581bb11ff645	count cells or regions × ascending or descending	The first demonstration separates every setting
0005ab5b7704	left to right or by density	Refute ranking all columns before selection

Table 5: Representative accepted parametric families.

9 Matching the original task

Generated inputs must cover the task they imitate. A distribution gate compares object count, component size, palette size and area with the original demonstrations. A profile floor fails a generator if no generated grid matches any original (height, width, palette) profile, or if its `self_check` rejects an original input.

10 The check battery

Labels are correct by construction once the rule runs as reviewed; `rule_smoke`, `authority_sync` and `selfcheck` protect that construction. Most other stages, like the rival rules, are regularizers on the data: they push a generator toward lessons a learner can use. A few judge how a Racket rule is written rather than what it computes. `racket_lint` enforces the house style, for example `(at g r c)` rather than `(cell-at g r c)`; nine of its thirteen checks are style or size rules. `grammar` asks whether the constrained decoding grammar admits every gold program, and `authority_sync` also fails when two copies of a rule differ only in text. Findings of this kind, about style, grammar coverage or text identity, matter only if the rules themselves become training data for writing programs. Once the submission could not train that in time, we stopped fixing them, and reviewer requests aimed at them were left open: a helper to synchronize behaviour-identical rule copies, a wider grammar, and fewer false alarms from the lint that flags Python names matching Racket primitives. Such a finding alone does not make a label wrong; the stages that check execution and the binding of approval to the reviewed source still apply. The twelve stages cover structure (`lint`, `override`, `racket_lint`, `grammar`), execution (`selfcheck`, `rule_smoke`), distribution (`dist_gate`, `profile_floor`, `diversity`) and semantics (`semantic_gate`, `defect_lint`, `authority_sync`). Each began as a defect reviewers found by hand.

Defect found in review	Check that now catches it
Generated median object count 12 against the original's 2	`dist_gate`
0% of generated grids match any original grid profile	`profile_floor`
Passed 120 of 120 execution runs while teaching the wrong key-to-direction mapping	`semantic_gate`
Ordinary draws that were identity transforms	`semantic_gate`
Gold program calls a helper the constrained grammar rejects	`grammar`
Generator crashes or emits constant outputs yet passes lint	`rule_smoke`, `selfcheck`

Table 6: Defects that produced checks.

Today's battery on the six sampled generators. The battery keeps growing, and approval is not re-run when it does. We ran today's twelve stages on the six generators of Section 5.3; none passes all twelve. All six pass `lint`, `selfcheck`, `rule_smoke`, `dist_gate`, `profile_floor`, `diversity`, `override` and `racket_lint`. All six fail `grammar`, a library-wide gate that fails because rules elsewhere use primitives the grammar does not list (`build-list`, `string-append`, ...), and `defect_lint`, whose newer `strict-scalar` detector finds that the generators accept invalid argument types (a boolean seed, a negative sample index, an unknown difficulty) instead of rejecting them. Neither changes the data drawn with valid arguments. Two fail `semantic_gate`: `5573cf8e2aa4` on its shortcut declarations (Section 5.3), and `5ca74a85bc85` because 4 of 12 raw lessons fit its declared rival, which automatic export rejects. `6b88648ff0e0` fails `authority_sync`: its two active copies of the rule differ in exact text, though they execute identically, and it declares no semantic contract, so its semantic gate is advisory.

Reviewing the reviewers. During the July review, an independent audit replayed reviewers' claims and published each gap as a numbered, mandatory correction. Every later review had to acknowledge all active items.

- **KRF-0001** (16 July): a tier-4 harness rating was not supported by a boundary cross-product probe. Overrides must be checked exactly, not merely accepted.
- **KRF-0002**: in `f2865c4fa81f`, 519 of 900 accepted `strict_subset=False` draws silently fell back to subset geometry, although the generic replay passed because some draws differed. The override contract now checks every accepted request against its observed value.
- **KRF-0003** (17 July): generators that passed every green gate still did not represent the task's manifold. This produced the semantic gate, whose contract was frozen on 24 July.
- **KRF-0004**: early-return code paths chose dimensions without the declared axis labels, escaping the axis-honesty check.
- **KRF-0005**: a healthy aggregate mutation-rejection rate hid one degenerate family that `self_check` still accepted. Each named impostor is now an individual obligation.

11 Review and approval

Review ran in two phases.

July–August 2026: every generator, one at a time. A queue covered all 3,937 generator modules then in the library; each item ended in an improvement or an evidence-backed "no change",

with ratings in six categories backed by a command, metric or file. 693 reviews were completed, by Codex, Kimi, OpenAI models and others, checking 4,890 shortcut hypotheses.

October 2026: reviewer packets. Reviewers, Claude and GPT Pro, receive packets for assigned generators and return repairs, extensions and evidence, which are integrated against the source hashes they were assigned. Appendix F quotes their instructions. 569 integrated review records are retained for 486 generators; older processed returns were compacted once their content was integrated.

Earlier October packets began with a blind solve of a separate approved task, before any reference material was opened; the reviewer wrote and revised a Python `solve(grid)` against the visible demonstrations, and every revision, wrong answer and timeout is kept. After the repair, the reviewer solved a fresh episode of the repaired generator, with its query answers scored privately after return. The final one-build packets assigned no solves. Every reviewer also answers a standing question: *name the strongest plausible rival rule that fits the original demonstrations, give an exact input on which it disagrees with the intended rule, say how the output was checked, and say whether the generator now emits such cases.* 629 approved generators keep a current answer, each with an exact input. For `bee63e6e52bd`, in which yellow pours from the gap in a frame and continues diagonally past both arm tips, the answer names *fixed two-cell mouth arms*: every original pair has arms of length 2 at both ends, so a rule that assumes 2/2 arms reproduces all four original labels, and a right-open 7×7 frame with arms of length 1 and 3 separates it from the intended rule (Figure 9). The generator now schedules arms of 1/1, 1/3 and 3/1. The library retains 2,638 solve attempts across 1,425 generators: 1,283 blind solves before repair (1,273 with no earlier exposure to the rule) and 1,355 after repair. By the reviewers' own reports, 2,499 attempts reproduce every visible demonstration. Maintainers scored the hidden queries of 104 attempts on 91 generators (10 blind, 94 after repair): 96 programs pass both the visible demonstrations and the hidden queries, 6 fail both, and 2 fail a visible demonstration yet answer the queries correctly. None fits every visible demonstration and then misses a hidden query. Such a failure would point to a lesson whose demonstrations admit a second rule; its absence does not show that the lessons are unambiguous, since a solver can pick the intended rule anyway. The other attempts are reviewer-reported and unscored.

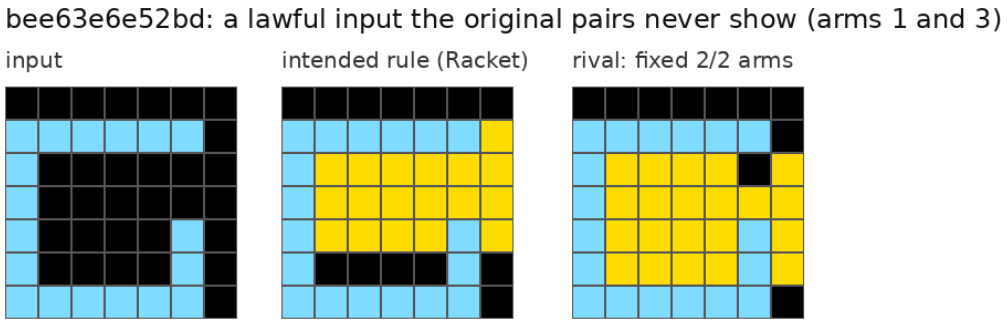


Figure 9: A reviewer's answer to the standing question for `bee63e6e52bd`, replayed: the input quoted in the answer, the reference rule's output, and the output of the declared rival that assumes both arms have length 2. Both rules reproduce all four original pairs.

Approval requires a `reviewed_safe` label recorded against the current generator and rule hashes; the labels and the full definition are in Section 15. Editing a reviewed generator revokes its approval

until it is reviewed again: 56 generators are unapproved because their code changed after review, and 15 because their rule did.

12 Comparison with RE-ARC, ARC-GEN and NVARC

Each dataset produces pairs (x, y) and certifies something about them. Stated precisely:

RE-ARC [5]. A hand-written generator G_t for each of the 400 ARC-AGI-1 training tasks emits $(x, y) = G_t(\omega)$; the pair is kept iff a separately written DSL solver V_t satisfies $V_t(x) = y$. Certified: two programs agree on each pair.

ARC-GEN [6]. A generator $G_t(\theta)$ emits (x, y) for 400 ARC-AGI-1 tasks and 500 ARC-AGI-2 tasks. Validation shows that for every original pair (x_i, y_i) there are parameters θ_i with $G_t(\theta_i) = (x_i, y_i)$. Certified: the original pairs lie in $\mathrm{Im}(G_t)$ (mimesis). Labels come from the same code that draws the input.

NVARC [7]. An LLM writes a task description, an input program g, and independent output programs $h_1, \ldots, h_{20}$; a puzzle is kept when at least 15 agree on its inputs [8]; 103k new puzzles were made this way and expanded to 3.2M augmented samples. Certified: a majority of independently written programs agree on every output.

ARCpuzzles. $x = G(\sigma, k)$ and $y = f(x)$ for a Racket reference rule f that reproduces every original pair. Certified, in addition: for the rivals R checked at export (those in `LESSON_COMPETITORS` and `CANONICAL_SHORTCUTS`; rivals declared only in `SEMANTIC_RIVALS` are checked by the semantic gate on sampled inputs, not per lesson), every lesson (D, q) exported with an automatic demonstration count, $D = \{(x_i, y_i)\}$, satisfies

$$\forall r \in R : \ (\forall i,\ r(x_i) = y_i) \ \Rightarrow \ r(x_q) = y_q,$$

and, in the semantic gate's sampled draws, every declared canonical shortcut r (a rival with $r(x_i) = y_i$ on all original pairs) has generated inputs x with $r(x) \neq f(x)$. With a fixed demonstration count a failing lesson is kept and flagged in its provenance; the fit of a declared shortcut is checked only by gates run after 23 October.

What "verified" means. NVARC and the 2026 third-place solution call their puzzles verified when independently written programs agree. We separate four properties a pair or lesson can have:

V1 consistent: two or more independently written programs give the same label.

V2 grounded: the labelling program reproduces every pair of the original task it imitates.

V3 identifiable: each lesson's demonstrations refute a declared set of rival rules on its query. Identification relative to all functions is impossible (Section 1), so V3 is always relative to a finite set of plausible rivals.

V4 reviewed and bound: a review is recorded against the exact generator file and the rule text (compared after whitespace normalization), and any change revokes it.

	V1	V2	V3	V4
RE-ARC	generator vs. verifier	verifiers reproduce originals (391 of 400)	–	–
ARC-GEN	–	generators reproduce originals	–	–
NVARC	≥15 of 20 programs	no originals (new puzzles)	–	–
ARCpuzzles (approved)	–	every reference rule	1,224 of 1,984	all 1,984

Table 7: Verification properties by dataset. ARCpuzzles does not use V1: labels come from one reference rule, checked by V2 and by execution. RE-ARC's 391 of 400 was measured during our training run. ARCpuzzles V3 counts generators that declare a lesson hook; the 941 parametric families are also certified per episode against their own settings.

In this paper, *verified* means V2 and V4 for every approved generator, and V3 for lessons that pass a declared lesson check. When the caller fixes the demonstration count, a failing lesson is kept and flagged `diagnostic_unsupported` rather than rejected.

What the trained data actually carried can be read from the export. In the round-38 training export, none of the 9,170 ARCpuzzles episodes was kept with a failing check: 5,156 are parametric episodes whose demonstrations leave their setting as the only one of the family's settings that fits; 3,035 passed a declared lesson hook; 314 passed declared rivals; and 665 (7.3%) come from generators that declare no check.

Screening RE-ARC, ARC-GEN and NVARC episodes. When the submission's training run added external episodes, each episode was screened for identifiability against a library of rivals: every other ARC-1 rule (all RE-ARC verifiers) and a set of rival templates (crops, components, compression, tiling and upscaling, under all eight dihedral views, literal or recoloured). The screen also required the output size and colours to be inferable from the demonstrations and rejected identity, dihedral and repeat shortcuts. An ambiguous episode received another demonstration, up to five, before it was dropped. One round's export recorded:

	RE-ARC	ARC-GEN	NVARC
Accepted episodes (generators or files)	3,717 (391)	6,099 (891)	855 (855)
Identifiable at the default demonstration count	3,639 (97.9%)	5,957 (97.7%)	824 (96.4%)
Needed one or two extra demonstrations	78 (2.1%)	142 (2.3%)	31 (3.6%)
Ambiguity triggers: rival template	239	98	9
another task's rule	72	123	6
output size not inferable	43	76	42
output colours not inferable	19	272	62
identity/dihedral/repeat shortcut	3	35	4
Queries dropped	68	220	19
Episodes rejected	50	129	18

Table 8: Identifiability screen applied to external episodes: RE-ARC and ARC-GEN in round r38, NVARC in round r35 (one episode per puzzle file). Rivals are a finite library run with a 0.3 s budget each; passing is evidence against these rivals only.

The same screen on ARCpuzzles episodes. During training the screen ran only on external episodes; ARCpuzzles episodes were checked against their own generators' declared rivals and hooks or, for parametric families, the settings certificate. Applied afterwards, unchanged, to 1,000 ARCpuzzles episodes drawn at random (seed 20261108) from the 9,170 in the round-38 export, with all four queries of each episode and no demonstration added:

	ARCpuzzles, round 38
Episodes screened (queries)	1,000 (4,000)
Every query passes	973 (97.3%)
Some query flagged ambiguous	18 (1.8%)
with 2 demonstrations	7 of 104 (6.7%)
with 3 or more	11 of 896 (1.2%)
Ambiguity triggers (queries): rival template	9
another task's rule (RE-ARC verifier)	4
output size not inferable	18
output colours not inferable	2
identity/dihedral/repeat shortcut	0
Hard flags (queries): output equals input	9
constant output	4

Table 9: The external screen of Table 8 applied after the fact to ARCpuzzles episodes. All four RE-ARC rules that fired belong to other tasks than the generator's own.

For the external libraries the screen acted on a flagged episode: it added demonstrations, dropped the query or rejected the episode. ARCpuzzles episodes were trained as exported, so about 1.2% of their queries carried a flag the external data would not have. Two-demonstration episodes of parametric families, which requested 2 to 6 demonstrations from 25 October, have the highest rate (6.7%); most flagged episodes have four demonstrations. Size and colour inferability are not part of the generators' declared checks, so those checks could not catch these triggers. The output-equals-input and constant-output queries come from generators that allow such outputs; the external screen rejects them.

Agreement between programs, whether two (RE-ARC) or fifteen (NVARC), constrains y given x. Identifiability constrains which x appear together in a lesson. They answer different questions, and agreement does not imply identifiability: the NVARC puzzle of Section 13 passes agreement and still yields lessons whose demonstrations admit a second rule. Reproducing the original task is not enough either: a canonical shortcut (Appendix E) fits every original pair by definition, so a generator's lessons must include inputs on which the intended rule and its shortcuts disagree.

13 Ambiguity found by blind solving

Label agreement certifies that a puzzle's outputs follow one rule. It does not certify that the demonstrations a learner sees determine that rule. To test this directly we audited puzzles from the NVARC synthetic set [7] with a commit–reveal protocol. A reviewer model (Claude Opus) sees the first four pairs of a puzzle and the input of the fifth, writes down a rule and a predicted output, and the hash of that prediction is recorded (`prediction-lock.json`) before the remaining pairs are opened. The prediction therefore cannot be revised after the answer is seen. The traces are retained and were never used for training.

A case with two correct rules. NVARC puzzle `ff72ca3e_7ddcd7ec` has 30 pairs. From pairs 0–3 the reviewer committed to: *from every blue cell, extend green rays along all four diagonals through black cells, stopping at the grid edge or the first non-black cell.* That rule reproduces all four visible pairs exactly. On pair 4 it differs from the true output in 23 cells (Figure 10), and on the other pairs it fails 16 more of 25, by 1 to 15 cells each. The later pairs reveal an interaction the first four never show: a ray that strikes an orange cell continues along the perpendicular diagonal. Both rules fit the four visible demonstrations; only one fits the puzzle.

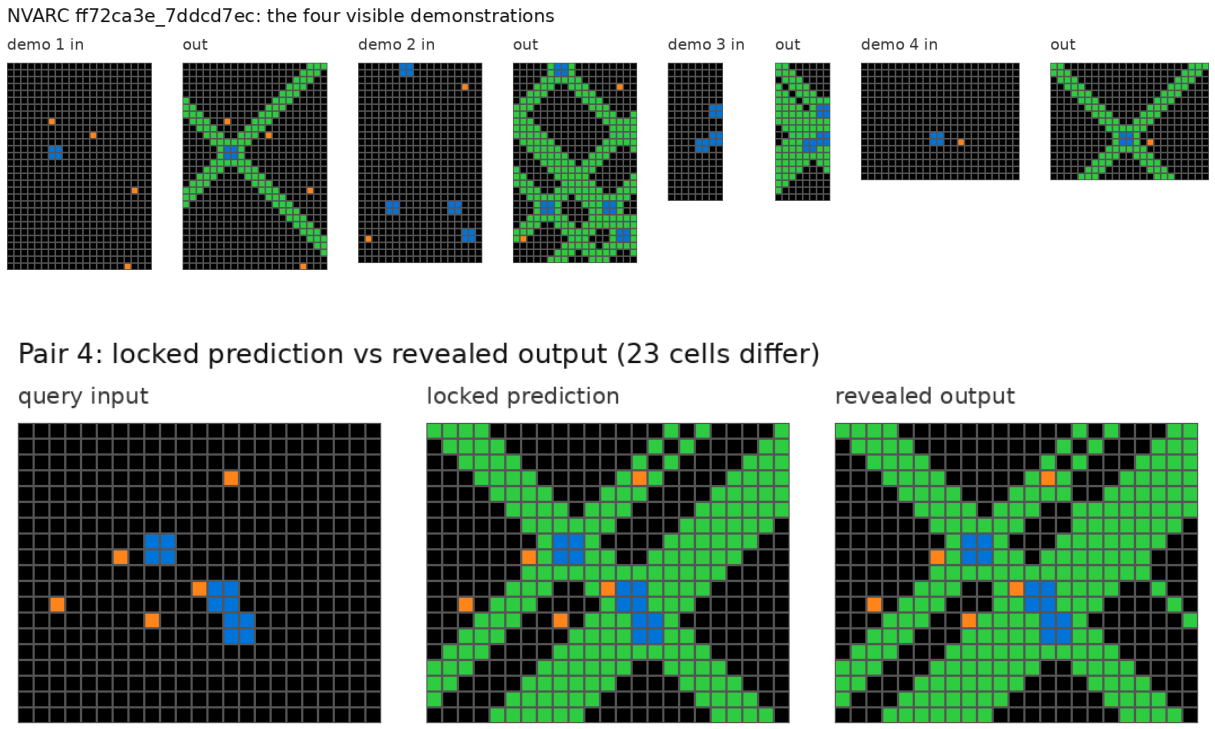


Figure 10: NVARC `ff72ca3e_7ddcd7ec`. Top: the four visible demonstrations. Bottom: the query input, the locked prediction, and the revealed output, which continues rays at orange cells.

How often a drawn lesson hides it. Let $c = 13$ of the puzzle's $n = 30$ pairs be consistent with the simpler rule and $b = 17$ not. If a training sample draws d demonstrations and one query uniformly without replacement, the probability that every demonstration is consistent with the simpler rule while the query is not is

$$P_d \;=\; \frac{\binom{c}{d}}{\binom{n}{d}} \cdot \frac{b}{n-d}, \qquad P_3 = 4.4\%, \quad P_4 = 1.7\%, \quad P_5 = 0.6\%.$$

Such a lesson is supervised to an answer its demonstrations do not determine. A generator with a lesson schedule (Section 5) instead places the distinguishing input in every window by construction, and `lesson_identifiable` rejects the lesson if a declared rival survives, when export chooses the demonstration count.

Scope. The audit covered 16 NVARC puzzles. Recorded verdicts: 9 consistent, 1 ambiguous (this case), 1 trivial (output independent of the rule's structure, pending external adjudication), and 5

without a parsable verdict. Sixteen puzzles do not support a population error rate, and we do not claim one.

Ambiguity in our own lessons, kept as a task. Reviews of ARCpuzzles generators found the same failure in our own lessons: a retained lesson whose demonstrations fit a second rule that changes the query answer. The generator was then repaired so that its schedule shows the distinguishing input, and the original lesson was kept. Such lessons form a separate auxiliary set, *find another valid answer*: the prompt shows the demonstrations, the query and one valid rule with its answer, and asks for a different rule that also fits every demonstration, and for that rule's answer. Both rules are Racket programs. A candidate is published only after both run, through the same protected bridge as the reference rules, on every demonstration and on the query, and are certified to fit every demonstration and to disagree on the query. The current snapshot holds 18 certified candidates from 10 generators: 17 lesson groups, 35 prompts, 160 Racket evaluations, none quarantined. The rule pairs are ones a solver can plausibly confuse: four- versus eight-connected components, enclosed versus small zero regions, at least one versus exactly one overlap, Manhattan versus Chebyshev or Euclidean distance, a column count versus the same count capped at two (Figure 11). The set was copied to the training host but never trained.

Ambiguity flagged while describing errors. The agents that wrote the refine sentences (Appendix B) also flagged answers whose demonstrations admit a second rule, at most 0.44% of those described in any library; the answers were the model's own attempts, so these are not population rates.

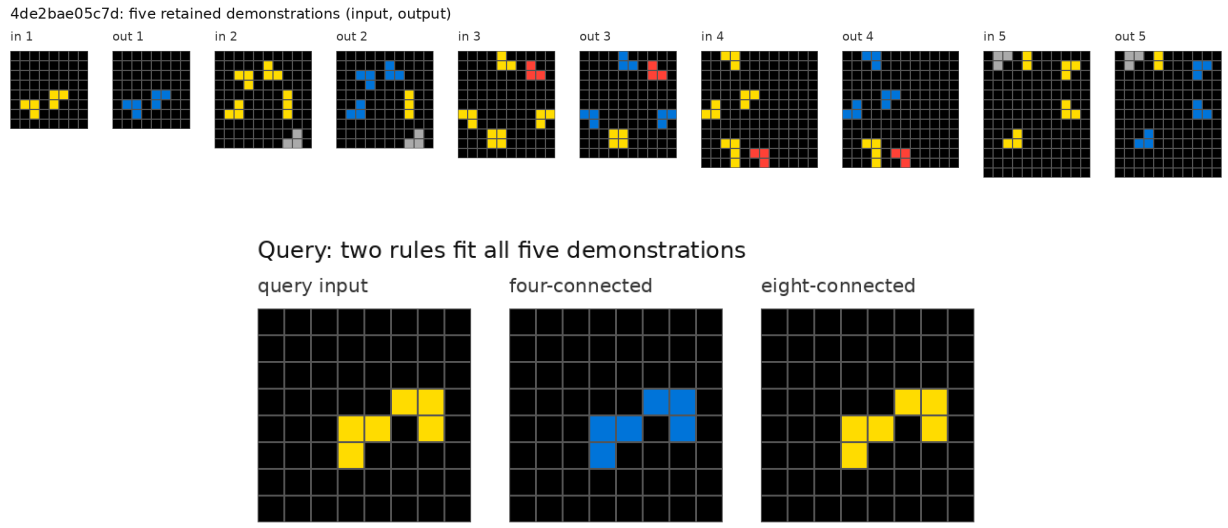


Figure 11: A retained pre-repair lesson of `4de2bae05c7d` (recolour yellow L-triomino components blue). Every demonstration fits both four- and eight-connected components. In the query two L-trominoes touch only at a corner: four-connected, they are two L shapes and turn blue; eight-connected, they are one six-cell component and stay yellow. The repaired generator makes every five-draw window include such a diagonal contact.

14 Ideas and pitfalls

We list the ideas we used and the pitfalls we hit, each with the case in this paper that shows it, to help others establish practice for generating this kind of data. It is what we learned, not a claim that the list is complete or that our choices are the best ones. Many of the ideas were chosen for efficiency rather than established by controlled comparison. On a budget the work is messier: we proceeded more like Bayesian optimization than logical deduction, trying the change that looked most likely to help given what we had seen, keeping it if the next measurement improved, and rarely spending compute to isolate its cause, since each controlled comparison costs about as much as the improvement it would test. Treat the ideas as reasonable starting points, not tested results.

Kinds of work. Building the data took several distinct kinds of work, done by the author and by AI agents:

Work	What it involves	Recorded amount
Writing rules	Racket reference rules to one written standard; parametric settings	4,061 generators; 941 families with 2,174 settings
Writing samplers	Python input samplers: axes, textures, invariants, schedules	one per generator
Attacking rules	Rival rules, witnesses, shortcut searches on the original pairs	6,822 rivals in 1,498 generators; 4,890 shortcut hypotheses checked in July
Reviewing and repairing	Review packets, repairs, new prototypes	693 reviews in July; 569 records in October (201 repairs, 291 new prototypes)
Solving blind	Solving a task before reading its rule, and again after repair	2,638 attempts on 1,425 generators
Auditing	Adversarial findings bound to the source they were found against	2,472 open findings on 1,748 generators
Building checks	The twelve-stage battery, the semantic gate, the lesson census	12 stages, each from a defect found by hand
Engineering	Shared runtime, export, provenance and hashing	about 3,500 commits, 26 March to 2 November
Describing errors	One checked sentence per model error, for training	17,537 answers

Ideas.

1. Compute every label by executing a reference rule; never author an output (Section 4).
2. Key each random choice by name, so that adding or changing one draw does not shift the others (Section 5).
3. Turn the sample index into a lesson schedule, so that every window of consecutive draws holds the inputs that refute the rivals (Section 5).
4. Write rival rules as executable functions, tie each to the input feature that refutes it, and reject a lesson that a rival survives (Section 7).
5. Search the original task's own pairs for shortcuts that fit them all, and require generated inputs on which each fails (Section 7, Appendix E).
6. Declare the textures that teach a rule and the textures that hide it (Section 6).
7. Make a rule parametric: one sampler, several executable settings, and a certificate that each episode's demonstrations leave exactly one setting (Section 8).
8. Replay original inputs at a known rate to anchor the distribution, and measure what that costs in effective size (Section 5.3).
9. Expose only exact, observable axes, and measure reach (distinct draws, Chao1, N_{eff}) instead of multiplying declared axes (Section 5.3).

10. Grow the check battery from defects found by hand (Section 10).
11. Bind approval to source hashes, and keep every finding attached to the source it was found against (Sections 15 and 11).
12. Start each review with a blind solve, committed before the reviewer reads the rule (Sections 11 and 13).
13. Keep ambiguous lessons as a separate "find another valid answer" task instead of discarding them (Section 13).
14. Keep answers in private files, apart from the model-visible challenges (Section 17).

Pitfalls.

Pitfall	Case in this paper	What helped
Advertised axes that do not vary anything observable	The imported `5573cf8e2aa4` declared nine axes, several singleton, aliased or inert. Declared configurations and measured reach disagree both ways: 3 declared and 99,997 distinct in 100,000 draws; 196 declared and exactly 2,376.	Expose only observable axes; measure reach
Imitating the originals teaches their shortcuts	Five of the six sampled generators have a rival that fits every original pair, test included, so it scores perfectly on the original task; every original of `fae242ae157d` uses areas 4/5/6 in scan order.	Rivals with generated counterexamples; mass where rule and shortcut disagree
Independent sampling can omit the distinguishing input	An NVARC puzzle hides its second rule from 4.4% of three-demonstration lessons; `5ca74a85bc85`, with no schedule, produced 4 ambiguous lessons in 60.	Lesson schedules; a per-lesson identifiability check
Fewer demonstrations, more ambiguity	The external screen flags 6.7% of our two-demonstration episodes and 1.2% of the rest.	A higher minimum count; certify every lesson
A certificate covers only the rivals it lists	The parametric certificate separates a family's own settings, not generic rivals such as a constant output size. In round 38, 17 of 801 two-demonstration parametric episodes (2.1%) show outputs of one size while a query needs another, against 5 of 4,355 (0.1%) with more demonstrations and 26 of 4,014 (0.6%) from ordinary generators. In one episode of `4dd0b3ed33ee` both outputs are 8×9 and three of four queries need 9×10 or 7×8.	Add generic rivals (constant size, constant colours) to every check
A fixed demonstration count bypasses rejection	With an explicit count the worker keeps a failing lesson and only flags it; the census, which fixes the count, kept 4 ambiguous lessons of `5ca74a85bc85`.	Read the lesson check in the provenance; drop flagged lessons
Subsampling sample indices breaks schedules	`5573cf8e2aa4` places a canonical anchor at every third index; its notes warn that a loader selecting one residue class of indices could bias the curriculum.	Keep indices consecutive
Replaying originals concentrates the distribution	`5ca74a85bc85` replays originals in 18.5% of draws: $N_{\text{eff}} = 146$ while Chao1 exceeds 62,000.	Measure N_{eff}; choose the replay weight deliberately
A declaration can be wrong	356 approved generators copy their whole rival list into `CANONICAL_SHORTCUTS`; `5573cf8e2aa4` and two of five others sampled fail the fit check.	Verify every declaration by execution
Approval does not follow a stricter check	The fit check arrived on 23 October; generators approved on 3 October and unchanged since fail it. None of the six sampled approved generators passes all twelve stages of today's battery (Section 10).	Re-run new checks on approved generators; record the check version with each approval

Pitfall	Case in this paper	What helped
An override can leave the rule's domain	`c4f6a0585dbf`: `palette_kind=warm` yields inputs its rule leaves unchanged.	Test every override path; keep audits open until resolved
A hidden table can ask for what no demonstration shows	`03478139591b` maps colours by a fixed table (3→1, 5→4, 8→2); a query whose colour no demonstration maps needs an output colour nobody has shown. Over round 38, 21 of 9,170 episodes (0.23%) have a query needing a colour absent from every demonstration output and from its own input.	Require each table entry a query uses to appear in a demonstration
Originals can break the target format	13 bank originals have a grid larger than 30.	Validate size limits at import
Label agreement is not identifiability	An NVARC puzzle passes 15-of-20 program agreement and admits two rules on its first four demonstrations.	Check labels and lessons separately
Grid text can merge under the tokenizer	With letter labels, `(R, C)` merges in 19,800 of 21,600 contexts.	Check token boundaries on every training example
Progress on trained generators overstates generalization	Mastery is measured on fresh draws of generators the model trained on; public-evaluation tasks were in training.	Keep held-out sets; record exposure

15 What is released

The data is large. The library holds 4,061 generators, against 400 in RE-ARC and about 900 in ARC-GEN; the 1,984 approved ones alone are about five times RE-ARC and twice ARC-GEN. 941 parametric families add 1,233 Racket rule variants on top. Each generator is a function of a seed and draws new instances without a fixed limit; Section 5.3 measures how many distinct instances six random ones reach.

We release all of it rather than a curated top subset, because on our budget most generators are work in progress, and their state is part of what the release documents: every generator, finished or not, with its label, notes, review evidence, counterexamples and tests. Users choose by label. The reviewer packets, their assignments and raw returns are not released; Appendix F quotes the packet instructions. The unfinished generators show every stage a generator passes through, from imported and unverified, to machine-checked, to reviewed and repaired, to approved, so the release documents the curation process and not only its output. They also show how much work remains to represent these tasks well: 2,077 generators are not approved, and 1,748 carry an open finding. Everywhere else, the paper describes only the 1,984 approved generators. The release is at https://github.com/definitelynotrussellkirk-bit/building-data-for-arc.

Item	Count
Generators (per-puzzle 4,012; generic 49)	4,061
approved for training	1,984
Canonical puzzle rows (original pairs, all variants)	4,472
Sources: puzzle bank, ARC-AGI-2 training and public evaluation, custom (Table 1)	4,061
Accepted parametric families (settings)	941 (2,174)
Reviewer solve attempts; standing-question answers	2,638; 632
Certified "find another valid answer" candidates	18

Table 10: Contents of the release.

Labels. Every generator carries a review label and a derived approval:

Label	Generators	Meaning
`reviewed_safe`	1,994	a reviewer rated training suitability 3 or 4 of 4, or a reviewer's repair was accepted
`reviewed_standard`	57	a reviewer rated suitability 2 of 4
`provisionally_safe`	584	a repair was returned, or machine checks passed (clean self-check, pinned rule hash, at least 95% valid draws) without review
`unverified`	1,007	no review, and the independent monitor gate is stale or missing
`unsafe`	415	an active semantic veto, a failed independent gate, or a machine suitability of 1 or less; mostly machine flags, not review findings
`unreviewed`	4	no record

A generator is *approved* when it is `reviewed_safe`, its generator file matches the reviewed SHA-256 exactly and its rule matches the reviewed hash after whitespace normalization, it has a canonical rule and a positive mix weight, and no semantic veto is active. Reasons for non-approval overlap: 2,067 are not reviewed safe, 56 changed code and 15 changed rule after review, 48 have no canonical rule, 15 have zero mix weight, 4 lack a reviewed hash and 2 carry a veto.

Open audits. A finding against a generator stays attached to the source it was found against until someone resolves it; a later edit does not clear it. 1,748 generators, 734 of them approved, carry at least one finding that is not closed. Of the 2,472 open findings, 1,674 await triage, 789 are unresolved and 9 need evidence. A finding does not revoke approval by itself. `arcpuzzles list -audit-unresolved` lists these generators and `show ID` gives each finding with its reproduction. For example, an audit of `c4f6a0585dbf` reproduced that one of its overrides (`palette_kind=warm`) yields inputs its Racket rule leaves unchanged.

Originals outside ARC's size limit. Thirteen bank puzzles, two of them with approved generators, have an original grid wider or taller than 30, so they are not valid ARC tasks and `show -challenge` refuses them. Their generators can still draw legal grids: `6c68e1b008a5`, whose original is 9×59, draws inputs at most 29 wide.

16 Exporting training data

`arcpuzzles export` samples only approved generators by default, either in proportion to each generator's mix weight or uniformly; seeds and the full pool are recorded. An episode can carry four supervised queries over one shared set of demonstrations, all under the same rule and setting. Query answers are written to private files, never to the model-visible challenge file. Each row records the generator hash and its verified original-task family, or marks the family unknown; no train/evaluation split is invented.

17 Using the release

Querying needs only Python 3.12 or later. Generation and checks also need Racket on the path (or `RACKET_PATH`), NumPy and Pillow. A portable bundle pins approved sources with a Linux Racket for machines without one.

```
python3 -m venv .venv && . .venv/bin/activate
pip install -e '.[runtime,dev]'
arcpuzzles stats                                  # counts and labels
arcpuzzles list --approved --source training --limit 20
arcpuzzles show 3ef9eae12446 --challenge          # demonstrations, no answers
arcpuzzles generate 3ef9eae12446 --count 2 --seed 17 --train 2 --test 1 \
    --out var/examples.jsonl
arcpuzzles export --approved --count 4 --seed 17 --sampling uniform \
    --style episodes --out var/datasets/approved-s17
arcpuzzles export --parametric-only --count 4 --seed 17 --sampling uniform \
    --style episodes --out var/datasets/accepted-s17
arcpuzzles inspect-export var/datasets/approved-s17
arcpuzzles check 3ef9eae12446 --battery --out var/checks/3ef9  # 12 stages
arcpuzzles lesson-census 3ef9eae12446 --per-band 20 --out var/census/3ef9.json
```

- **Train on approved generators.** Generation and export refuse unapproved generators by default. `-allow-unapproved` opens them for research and labels every output as unapproved.
- **Keep answers private.** Exports write the model-visible challenges and the private labels to separate files; the labels must never enter the challenge file. `public.jsonl` holds every query input of an episode, so keep it with the controller and show the model the demonstrations and only the current query (`iter_public_turns`; `docs/datasets.md`).
- **Read the provenance.** Every episode carries its seed, sample indices, executed rule hash, rejections and lesson check. With a fixed demonstration count, drop episodes whose check is `diagnostic_unsupported`.
- **Keep sample indices consecutive.** A lesson schedule is a function of the sample index; a loader that keeps one residue class of indices can remove the inputs that refute the rivals.
- **Check open audits** before relying on a generator, and decide whether its finding matters for the use at hand.
- **To continue a generator**, start from its `notes.md`, counterexamples and tests, run the battery, and treat its label as a claim about the reviewed bytes only.

18 Submission

Our ARC-AGI-2 submission is a Qwen3.8-27B that writes the output grid directly, trained for 273 GPU-hours (231 on one GPU, 42 on two rented multi-GPU pods) on training examples drawn almost entirely (94%) from generators: ARCpuzzles plus screened RE-ARC, ARC-GEN and NVARC episodes. The run, its supervision format and its evaluations are in the appendices (A–D). Submitted alone, the model scored 5.00 on the public leaderboard (submission 57783445, 29 October); three other submissions of it between 27 and 30 October scored 2.50 to 3.75. Our recipe saw only 379,598 training examples over 48 rounds. NVARC's solution trained on 3.2 million augmented samples of 103k synthetic puzzles [7], and the 2026 third-place solution on about six million verified puzzles, each expanded into 24 augmented samples [8]. We do not attribute the accuracy gap to data volume alone; model, format, test-time training and compute differ.

19 Against the judging criteria

The paper track scores six criteria. We answer each directly.

Accuracy. Low. The model trained on these data, submitted alone, scored 5.00 on the public leaderboard (submission 57783445); three other submissions of it scored 2.50 to 3.75 (Section 18). Our local public-evaluation scores are not independent evidence, because those tasks were in training (Appendix C).

Universality. None of the steps depends on ARC grids; only the DSL and the samplers do. Compute every label with an executable rule; draw inputs with keyed, reproducible randomness; declare the rival rules a learner might infer; schedule inputs that refute them; for parametric families, certify per lesson that only one setting fits; and bind approval to the reviewed source. Any task family whose rules can be executed and whose inputs can be generated can follow the same steps. Our evidence covers ARC-style grid tasks only.

Progress. Others can use three things to reach a higher score: the data, with 1,984 approved generators and 941 parametric families adding 1,233 rule variants, released with the 2,077 unfinished generators and labels that say how far each got (Section 15); checks that test a lesson against declared rivals, and a screen that flags ambiguous episodes in RE-ARC, ARC-GEN, NVARC and our own data (Section 12); and the ideas and pitfalls from building them (Section 14). We have not shown that these choices raise a leaderboard score. Our own run suggests that the next step was training longer, not adding capacity (Appendix A).

Theory. An executable rule makes every label correct, but correct labels can still leave a rival rule that fits every demonstration and answers the query differently (Section 1). A good lesson is lossless on a class of sensible rules yet hard to invert. Declared rivals make that class concrete; schedules place the inputs that refute them in every window of consecutive draws (Section 5); lesson checks and per-episode certificates reject lessons the rivals survive (Sections 7 and 8). Agreement between label programs, the check RE-ARC and NVARC use, constrains each output given its input, not which inputs appear together, so it cannot provide this (Section 12). That such lessons train better models is a hypothesis we did not test with a controlled comparison.

Completeness. The appendices give the model and adapter configuration, the optimiser and its changes, the data mix by source and mode, the supervision formats, the evaluations, and the prefill scans that chose the data (Appendices A–D). Section 17 gives the commands to regenerate data from the release.

Novelty. Compared with RE-ARC, ARC-GEN and NVARC, the datasets we examined, the new elements are: treating the identifiability of each lesson, not only label agreement, as a requirement; declared rivals with schedules and per-lesson checks; parametric families certified per episode; review and approval bound to exact source hashes; and releasing unfinished generators with labels that state their stage (Section 12). We make no claim beyond those three datasets.

20 Conclusion

Labels computed by an executable rule are correct by construction, so the hard part of building data for ARC is the lesson: which inputs appear together, and whether they rule out the readings a solver would plausibly take instead. No finite lesson determines a rule, so every lesson check here is relative to declared rivals, a regularizer rather than a proof. Within that frame, the measurements in this paper say where lessons fail. Simple shortcuts fit the original pairs of five of six randomly

sampled generators. In one audited NVARC puzzle, independent sampling hides a second rule from 4.4% of three-demonstration lessons. Two-demonstration episodes are flagged several times as often as longer ones. Declarations and approvals drift as the checks tighten: none of six sampled approved generators passes today's battery in full.

We release all 4,061 generators, unfinished ones included, with their labels, open findings and evidence, so that others can use the approved ones, see how much work remains, and continue it. Section 14 lists the ideas and pitfalls we would want to know before starting again.

21 Limitations

- Rival and identifiability checks cover declared rivals on sampled lessons; they do not prove uniqueness.
- Approval binds a generator's source hash, not the gate version. Checks added later, such as the 23 October verification of declared canonical shortcuts, are not re-run on approved generators; three approved generators examined here fail it.
- ARCpuzzles episodes were not filtered by the external identifiability screen of Table 8 during training. Applied afterwards to a random 1,000 of them, it flags an ambiguous query in 1.8% (Table 9).
- Most review results are reviewer-reported and were not re-run when integrated.
- The 415 `unsafe` labels are not a review-measured defect rate. A generator is `unsafe` when it has an active semantic veto, failed an independent monitor gate, or, before any review, has a machine-estimated suitability tier of 1 or lower. Unsafe generators are excluded from training exports.
- 120 generators were written for ARC-AGI-2 public evaluation tasks, and the submission's training run included 3,981 training examples from public evaluation tasks. Local public-evaluation scores of that model are therefore not held-out measurements; the competition's hidden test tasks are separate.

References

[1] R. Kirk. What's all the Racket about? ARC-AGI-2 generators, v0.1. https://github.com/definitelynotrussellkirk-bit/arc-agi-2-generators, May 2026.

[2] F. Chollet. On the Measure of Intelligence. arXiv:1911.01547, 2019.

[3] F. Chollet et al. ARC-AGI-2: A New Challenge for Frontier AI Reasoning Systems. arXiv:2505.11831, 2025.

[4] ARC Prize 2025: Technical Report. arXiv:2601.10904, 2026.

[5] M. Hodel. Addressing the Abstraction and Reasoning Corpus via Procedural Example Generation. arXiv:2404.07353, 2024. https://github.com/michaelhodel/re-arc.

[6] ARC-GEN: a mimetic procedural benchmark generator for the Abstraction and Reasoning Corpus. arXiv:2511.00162, 2025. https://github.com/google/ARC-GEN.

[7] I. Sorokin and J.-F. Puget. NVARC solution to ARC-AGI-2 2025. https://github.com/1ytic/NVARC.

[8] Darragh and CPMP. [ARC-AGI-2] Public 3rd Place Solution. Kaggle, 2026. https://www.kaggle.com/competitions/arc-prize-2026-arc-agi-2/writeups/arc-agi-2-public-3rd-place-solution.

A The submission training run

Direction was set on 22 October 2026: one Qwen3.8-27B answering with the grid directly, thinking disabled; everything trains, with no hold-out except generators approved after a round was frozen; and no puzzle trains twice, so each training example is a fresh generator instance. A *route* is one training example: a prompt and its supervised answer turns, in one of the modes of Appendix B.

Model	Qwen3.8-27B, LoRA rank 128, thinking off
Optimiser	one AdamW kept across rounds: LR 10^{-4}, betas (0.9, 0.95) to round 13; 1.5×10^{-4}, (0.9, 0.98), 28 routes per update from round 14; 7×10^{-5}, 112 routes per update and $\beta_2 = 0.99$ by round 25 (decided 28 October, when training loss had flattened)
Schedule	48 continual rounds, 22 October – 2 November 2026
Training time	231.4 GPU-hours on one RTX PRO 6000 across 42 rounds, plus 41.6 on rented 4- and 8-GPU pods
Global updates	about 19,970 by 2 November
Round datasets	379,598 admitted routes
from generators	358,387 (94.4%)
from public training tasks	17,230
from public evaluation tasks	3,981
Routes by mode	direct 207,625; refine 70,092; verify-then-fix 55,461; verify 27,304; points 14,079; fix 3,597; exec 1,440
Parametric families	from round 7; up to 292 families and 623 settings in one round
External generators	RE-ARC 391, ARC-GEN 891, NVARC 855 files, requested at one third of generator items
Pilot (round 1)	4,332 episodes from 1,459 approved generators in 530 families; answer loss 0.191 $\rightarrow$ 0.124

Table 11: The training run.

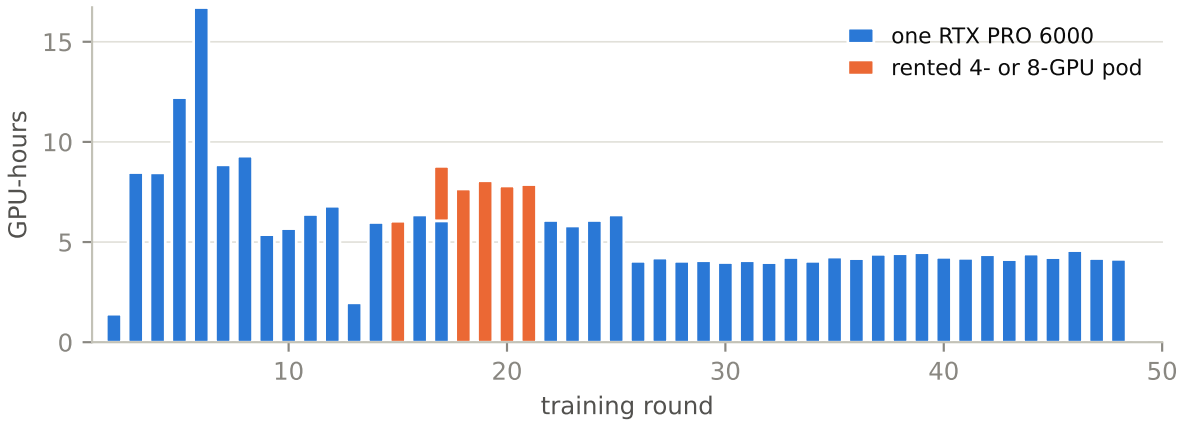


Figure 12: GPU-hours trained per round. Blue: the single RTX PRO 6000. Orange: rented pods, as GPUs times trainer seconds per round, round switches excluded (40.1 GPU-hours; 41.6 including switches). Round 17 was split between the two. Round 49 was still training.

Burst runs on rented GPUs. The main run trained on one RTX PRO 6000 (Blackwell, 96 GB). On 26–27 October we tried three rented bursts that continued the same adapter and optimiser state: one B300 was stopped by its cost guard, a 4-GPU pod finished one round, and an 8× RTX PRO 6000 pod trained for about five hours (4.45 hours of training, rounds 17 to 21) at six times the single GPU's speed. Its prepared rounds ran out before its time stop: we did not know how much data, or how much time, a longer run needed.

We decided against further burst runs, because we could not tell how much the model's ability was still increasing per round, and because we were not sure we would be able to submit.

Was the run long enough? Two measurements, taken afterwards, suggest that adapter capacity was not the limit and that training longer would have been the cheapest next step.

- **Capacity was barely used.** In the final adapter (update 19,969, rank 128), the 32 largest singular values of each weight update hold a median 98.2% of its squared Frobenius norm (its energy), and at least 88.2% in every one of its 496 tensors; the median stable rank is 2.5. A rank-32 adapter could have held almost all of what this one learned.
- **Training had not converged.** The adapter's norm was still growing, from 10.1 at update 13,673 to 14.7 at the end. Each route is trained once, so a round's training loss is measured on examples the model had not seen. That loss fell from 0.032 in rounds 38–40 to 0.025 in rounds 48–50. The repair modes fell (verify 0.0057 to 0.0030, verify-then-fix 0.048 to 0.042), and verify-then-fix routes also made up less of the mix (56% to 48%). The direct-answer loss held near 0.017 over the same rounds, on data chosen as the most surprising draws available.

Training for about two weeks (22 October to 2 November) sounds long, but it is short by current standards, and keeping one Colab session alive for the whole run was itself difficult.

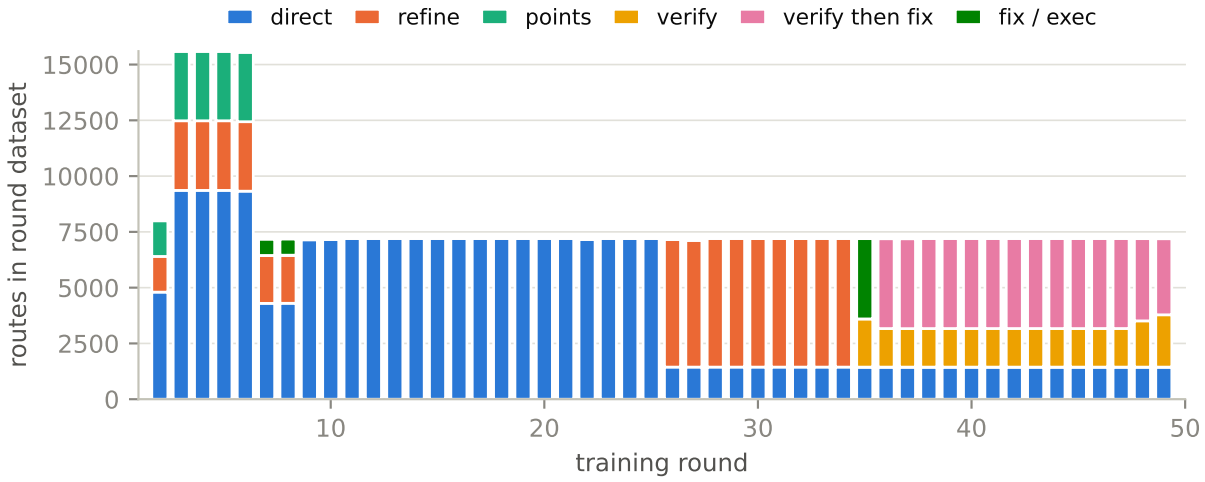


Figure 13: Routes per round dataset by supervision mode. Rounds 2–6 mix direct answers, refinement of the model's own earlier attempts, and point queries; rounds 7–8 replace point queries with Racket execution routes; rounds 9–25 are direct only; rounds 26–34 are mostly refinement; round 35 splits repair into verify and fix; from round 36 the model first verifies a previous answer and, if it is wrong, writes the whole correct grid.

B Supervision format

Supervision changed several times during the run (format versions 2 to 8). The final format has these parts.

Labelled grids. Every grid begins with a name line (`train 1 input, size 26x21`). The next line is a column ruler: `i` for an input or `o` for an output, a bar, and one label per column. Every row begins with its row label and a bar. Labels run from 0 to 29 using one character each, so that each label is one token and rows and columns stay aligned: `0123456789aBcDfEgHjIqkLpNtOvUw`. A cell is written `~R|C` for row label R and column label C; `~c|4` is cell $(12, 4)$. Labels were added on 23 October so the model can locate cells instead of counting rows; the one-token constraint followed on 24 October.

Nothing merges under the tokenizer. Every character of a size line, a column ruler, a labelled row, a cell reference and a label list is exactly one token, in every context where it appears. The build checks this on the stored token ids of every training route and evaluation prompt and fails on any violation. The Qwen tokenizer keeps the digits 0–9 separate, so each cell of a row is one token. Letters merge with their neighbours, so the twenty letter labels were chosen, in mixed case, so that no two adjacent labels merge; this was checked for all 900 row–column label pairs. The base model itself writes cells as decimal tuples, $(12, 4)$, which costs seven tokens; with letter labels every bracket, comma or space form merges a label into its neighbour. Measured with the pinned tokenizer over all 900 label pairs, 8 preceding contexts and 3 joins:

Cell notation	Labels that stay one token
`(R, C)`	digits only; 19,800 of 21,600 contexts merge
`(R,C)`	digits only; `,a` merges
`(R|C)`, `[R|C]`, `(R;C)`	digits only; the opening bracket merges with letters
`~R|C`	all 30, unless a space precedes `~` (the renderers never write one)

`~R|C` was chosen for exactly this reason: it is the one form in which no label ever combines with its neighbour. `~` plays the opening bracket and `|` the comma, keeping the model's own row-first frame with tokenizer-safe glyphs. Of all printable ASCII, Greek letters and mathematical symbols, only the digits stay separate after a bracket, a space and a comma; the only alphabets that would survive the literal tuple are CJK ideographs and fullwidth forms, which carry number meanings that clash with the colour digits. The ruler corner is a lowercase `i` or `o`, because uppercase `I` and `O` are labels 19 and 26 and both stay one token before `|` (`>` would merge into `>|`). Audits on 24 October found no violation in 1,371,787 and 2,937,294 labelled spans of two rounds, and none in 1,443,329 spans of the first dataset in the final grid format.

Size and colours first. An answer states its size, then the colours it uses, then the ruler and rows. Committing to the output size before any cell is written was the first format decision (22 October): reading left to right, every later token is conditioned on a declared shape.

Facts line. After each test input, a line computed from the shown grids gives the input's size, colour counts and same-colour shape counts, then what the train pairs share: whether the size is preserved, the output colours, colours added or removed, changed cells per pair, the background, and symmetry. It is about 90 tokens and is the same for every task, so the model has a fixed place to read structural facts.

Answer facts: extra supervision. The facts were added for extra supervision. From 23 October, half of the direct and point routes also ask, at the end of the user turn, for 2 to 4 facts about the answer, and the target adds a `facts` block after the grid or cells: the count of one colour, that colour's count in each row or column, a colour's bounding box as two cell references, and similar. Each fact is a name line, then its value, in the same names and formats as the Facts line and the labelled grid. Values that hold labels start their own lines, because a preceding space or comma would merge a letter label into another token. The request never hints at the answer: colour facts name only colours the prompt shows, and the size and colours that every answer already states are never asked. The request is part of the prompt so that the model is conditioned to expect a facts block only when asked. We never planned to ask for facts at inference; with the request in the prompt, the extra targets do not teach answers to continue past the grid, and a prompt without a request keeps the same format at training and at inference. Facts tokens were 3.5–6.4% of supervised tokens in rounds 6–10, were switched off in rounds 11–25, and were 11–17% from round 26 (3% in round 35).

Four queries per prompt. Only answer tokens carry loss, so a prompt whose demonstrations are long and whose answer is short supervises little of what it processes. Rounds 2–4 answered one test input per prompt. From round 5 (23 October) the round builder asks every direct route for four test queries on one set of demonstrations, given as successive user turns: the demonstrations are read once and supervise four answers. A route carries fewer only when the episode's remaining test pairs were already trained, are held for point queries, or would exceed the token budget. Rounds 4

and 5 have the same mode mix (9,360 direct, 3,120 refine and 3,120 point routes) and the same 15,600 segments, so they isolate the change:

	round 4	round 5
Queries per direct route	1	4 (88% of routes)
Direct answers supervised	9,360	35,828
Tokens processed	28.6M	42.7M
Tokens supervised	2.44M	6.54M
Supervised share	8.5%	15.3%

Processed tokens grew by 1.5× and supervised tokens by 2.7×. In the direct-only rounds 11–25 the supervised share was 21–24%. It falls to about 12% from round 26, when most routes are refine or verify routes, which show a previous answer and supervise one reply.

Modes.

Direct. Demonstrations and a test input; the target is the full answer.

Refine. A previous answer is shown; the target is the correct grid, or the same grid when it was already right. Previous answers are always the model's own real attempts on generator puzzles, never synthetically perturbed gold. From round 26 the repair first locates the errors. In round 26 it listed the wrong cells, then their colours. From round 27 an `error` block opens with the output size, then says `no error`, or `wrong size`, or lists every wrong cell as `~R|C` in row order and only then their count; the whole correct grid follows. Listing the cells before the count, and repeating the full grid after them, gives the model a second chance to emit the grid in order. Before the fix, the target can carry one English sentence on the cause of the error. Claude agents (Opus from 24 October) that see the puzzle's rule, the answer and its wrong-cell map pick one of ten causes (wrong target, missed cells, wrong shape, wrong colour, overreach, wrong size, no change, wrong rule, correct, ambiguous) and write the sentence to a strict guide: colours as `color N`, cells by their row and column labels, counts in digits. A checker rejects any sentence off the guide. 17,537 of the model's answers were described this way. When another rule also fits every demonstration, the cause is *ambiguous* and the sentence names it (*. . . but another possible rule is . . .*), so the model learns to name a second reading it can use for its second attempt.

Points. The prompt lists 9 to 16 cells, or a 3×3 to 4×4 block, and asks for their values. Since 24 October the answer first states the output size, then the values. Cells are chosen from the hardest available: cells the model got wrong in its attempts, else the highest-loss cells from a prefill scan (Appendix D), else cells that differ from the input.

Verify, fix, verify-then-fix. Round 35 split repair into its two tasks: *verify* shows a previous answer and asks only for the `error` block; *fix* shows a previous answer with its `error` block and asks for the whole correct grid. From round 36 both share one prompt: the first turn verifies, and when the previous answer is wrong a follow-up turn asks for the whole correct grid, so one reading of the puzzle trains both tasks. A right previous answer stays a single-turn verify.

Exec (rounds 7–8). The demonstrations are replaced by one short instruction and the generator's executed Racket rule, byte for byte; the generator's own inputs follow one per turn, and each answer is the output grid in the ordinary answer block. No examples and no Facts line. 720 routes per round, each from a different generator. The rule is given losslessly as a program, and the short prompt raises the ratio of supervised to processed tokens. Dropped from round 9, when the mix became all direct.

A point query from round 6, with the prompt's grids omitted:

````
Facts: test 24x25; counts 5:598 8:1 9:1; objects 5:1 8:1 9:1. demos 4;
size rule same; out 0 1 3 4 5 6 7 8 9; added none; removed none;
changed 24 14 20 18; bg 4; symmetry none.

Cells:
~k|7~k|8~k|9~L|7~L|8~L|9~p|7~p|8~p|9
--- target ---
```cells
24x25
9 5 5 5 5 5 5 5 5
```
````

The start of a direct answer from round 48:

````
Facts: test 1 input size 26x16; counts 0:176 1:2 2:2 6:2 7:180 8:2 9:52;
shapes 1:2 2:2 6:2 7:15 8:2 9:15. train pairs 4; size rule same; output
colors 0 1 2 3 4 5 6 7 8 9; added none; removed none; changed cells per
train pair 10 4 4 6; bg 0; symmetry none.
--- target ---
```grid
size 26x16
colors: 0 1 2 6 7 8 9
o|0123456789aBcDfE
0|0000000000000000
1|0777707777077770
2|0729707297072970
...
```
````

C Late-round evaluations

Adapters from the last training days were evaluated on Slurm with one sample per item. Public evaluation tasks were in training from round 2, so that panel is not held out. Fresh panels draw new instances from generators, so no two evaluations score the same items; as with the scans, we chose not to keep a fixed held-out set (Appendix D). From 28 October the fresh panels also fed repair training: the model's answers to them became repair examples (Appendix B), so an evaluation doubled as data, and only the public-evaluation panel stayed fixed. Retention re-asks 48 routes sampled at random from the round's own training data, so it measures whether trained items are reproduced.

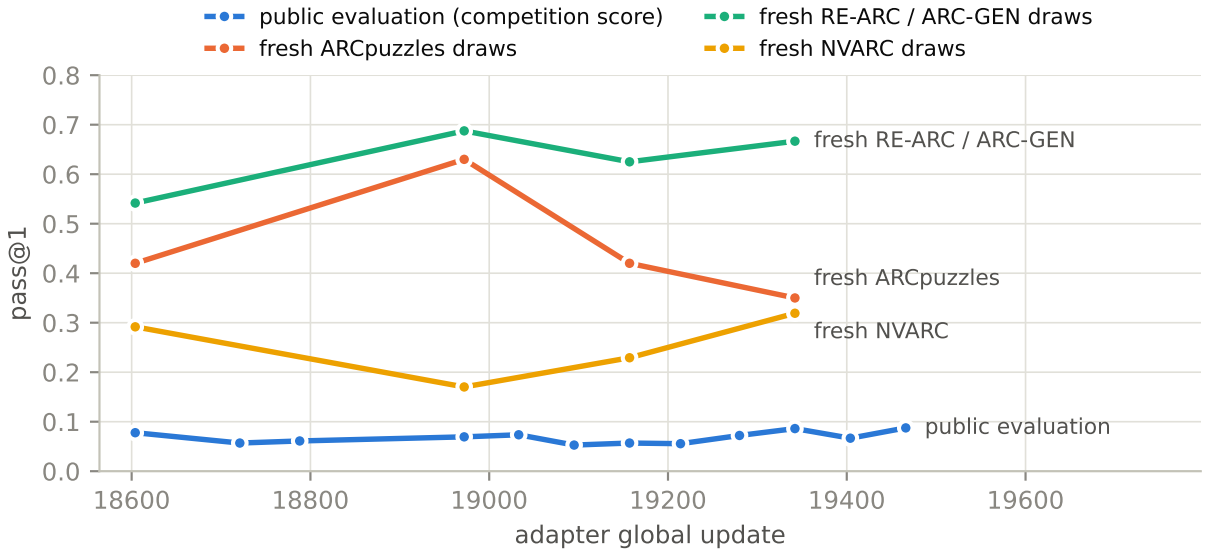


Figure 14: pass@1 by adapter. Public evaluation is the competition-style score over 120 tasks (any of up to two attempts; here one sample).

D Prefill scans and per-generator mastery

Scanning by prefill. To find what the model had not learned without sampling its answers, the gold answer was scored in one forward pass. For a prompt p and gold answer tokens $a_1, \ldots, a_T$ the scan reads

$$q_t \;=\; P_\theta(a_t \mid p,\, a_{<t}), \qquad t = 1, \ldots, T,$$

from the prompt log-probabilities of the concatenation $p\,a$, and generates nothing. An item is *mastered* when $\min_t q_t \geq 1/2$. Then no other token is more likely than a_t at any step, so greedy decoding reproduces the gold answer (barring an exact tie at $1/2$). Until 23 October the rule was $\prod_t q_t \geq 1/2$, which marks a long answer hard even when every cell is confident. Each grid cell is one token, so the q_t of the answer rows form a loss grid over the cells; a cell with $q_t < 1/2$ fails the threshold, although its gold colour may still be the greedy choice given a correct prefix.

The scans ran in two lanes. The training cluster served the BF16 model with vLLM and asked for prompt log-probabilities with a one-token completion. Local RTX 3090 and 4090 cards ran an IQ4 GGUF quantisation through a small C++ program on libllama (`prefill_nll`), at about 870 prefill tokens per second on the 3090. A scan costs one forward pass per item and no decoding, so the local cards scored many more items than they could answer. Scans targeted the next round's export before it trained, so scanned draws were untrained.

No fixed evaluation set. We kept no fixed held-out set for measuring progress. Each scan scored draws that the next round would then train on, so one forward pass per item both measured the model and chose what to train, and no compute went to items that would never be trained on. Several other choices in the run made the same trade of consistency for efficiency: prefill scores instead of sampled answers; quantised IQ4 lanes on local cards beside the BF16 lane; four test queries on one set of demonstrations (Appendix B); and, in rounds 7 and 8, executed rules in place of demonstrations, which shortened prompts. The cost is comparability. Scans at different

checkpoints score different items, and the two lanes quantise the model differently, so mastery rates from different scans are not a benchmark over time; the scoreboard therefore classes each generator by its newest observations.

How the scans steered the data.

- **Draw order.** The round builder scored each generator draw by gold-answer loss per processed token: the draw's own scan, else its generator's newest scanned mean, else the median. Mastered draws went to the back of the queue, except a seeded 10% retained; every generator's first kept draw came before any second. A revised generator (new source hash, or new executed rule hash for a parametric setting) did not inherit its old score. Round 38 scored 189,689 draws, kept 12,633 generator draws, moved 2,446 mastered draws back and retained 273.
- **Point queries.** Without a wrong attempt of its own, a point query asks for the scan's highest-loss cells (Appendix B).
- **A per-generator scoreboard.** Every scan and every greedy answer to an untrained draw was merged into one record per generator.

The scoreboard. By adapter update 18,604 (2 November) it merged 138 scans with 252,766 scored items and 141,432 greedy answers. A generator's class uses its newest three scan items and newest three answers:

mastered: at least two recent scan items, all mastered, and no recent wrong answer (with no scan items: at least two recent answers, all right);

easy: a recent mastered share or exact-answer share of at least one half;

hard: no recent mastery, and on average at least 15% of answer cells with $q_t < 1/2$ (or at least two recent answers, none right);

unsolved: at least three observations, never mastered and never answered right;

medium: everything else measured.

Library	Generators	Mastered	Easy	Medium	Hard	Unsolved	Unmeas.
ARCpuzzles (approved)	1,984	473	843	268	305	85	10
written for public-eval tasks	120	2	18	21	45	34	0
RE-ARC	391	89	166	52	77	7	–
ARC-GEN	871	310	326	115	91	29	–
NVARC (by source task)	836	44	152	596	20	24	–

Table 12: Scoreboard classes on 2 November. Classes are measured on fresh draws of generators the model trained on: "mastered" means new instances of a trained rule, not a held-out rule. NVARC files are one-offs and count under their source task. The median ARCpuzzles generator has 115 observations.

A quarter of the approved ARCpuzzles generators (473) were mastered and two thirds (1,316) mastered or easy; 390 were hard or unsolved. The 120 generators written for public-evaluation tasks were the hardest group: 79 hard or unsolved, 2 mastered. The six generators sampled in Section 5.3:

Generator	Class	Scan items	Mastered (all items)	Greedy answers right
`3ef9eae12446`	mastered	29	96.6%	–
`7ce16af847d6`	easy	95	54.7%	55/80
`fae242ae157d`	medium	56	21.4%	25/40
`5573cf8e2aa4`	mastered	68	86.8%	42/43
`5ca74a85bc85`	mastered	61	77.0%	26/33
`6b88648ff0e0`	easy	85	35.3%	31/43

Table 13: Scoreboard records of the six sampled generators. "Mastered (all items)" counts every scan item since the generator was first scanned; the class uses the newest three. For `6b88648ff0e0`, 24 unmastered items had no cell below $q = 1/2$: the miss was in the size or colours line.

What the model found hard. Hard and unsolved generators usually fail narrowly. In ARCpuzzles, RE-ARC and ARC-GEN alike, the median hard or unsolved generator has 0.9% of its recent answer cells below $q = 1/2$: given the correct cells before it, the model gives nearly every gold cell probability at least 1/2, yet a few cells fail and its greedy answers are rarely exact. The NVARC median is 16.3%: NVARC files received no greedy answers, so an NVARC file is classed hard only through the 15% cell threshold, while in the other libraries a generator is also hard when its recent answers were all wrong. Table 14 lists the five ARCpuzzles generators with the largest recent share (among those with at least 8 scan items, not counting those written for public-evaluation tasks), all from the puzzle bank; each asks the model to count or compare objects and write the result into a small output grid (Figure 15).

Generator	Rule	Class	Scans	Answers right	Cells $q < 1/2$
`941f7ac13580`	count each object's cells; output the counts in reading order	unsolved	18	–	60.0%
`b3750af4dac1`	find the room whose area, doors and colour counts match the first room's; mark it in a strip	unsolved	54	–	44.4%
`6c2353a2cdee`	rotate three templates by their codes, overlay them, colour each cell by how many cover it	unsolved	78	0/81	28.0%
`71982ba2b59e`	$N \times N$ matrix relating every pair of objects: same shape up to rotation, same colour, or neither	hard	83	1/69	17.8%
`26f313fdf812`	$N \times N$ matrix comparing the areas of every pair of objects	unsolved	85	0/64	12.0%

Table 14: The five ARCpuzzles generators with the largest recent share of answer cells below $q = 1/2$, among hard and unsolved generators with at least 8 scan items, not counting those written for public-evaluation tasks. "Answers right" counts every greedy answer to an untrained draw; "–": none was requested.

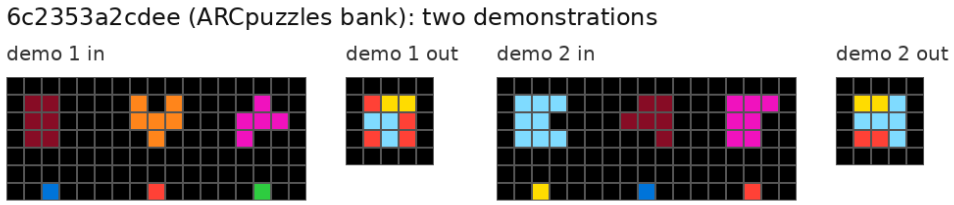


Figure 15: `6c2353a2cdee`, seed 17. The bottom row holds a rotation code (colours 1–4) under each template; the output overlays the rotated templates and colours each cell by how many cover it (1 red, 2 yellow, 3 cyan). The model answered none of 81 draws right.

The same ARC task can be hard in one library's version and easy in another's. 26 ARC training tasks are hard or unsolved in at least two of RE-ARC, ARC-GEN and NVARC; some are easy in ours (for example `36d67576`), and some are hard everywhere (for example `dc2aa30b`, where the model answered 0 of 58 ARC-GEN draws and 1 of 68 of ours right). The versions differ in how inputs are drawn and in how much of each the model saw, so these gaps do not measure lesson quality.

E Rivals that fit every original pair

A canonical shortcut is a rival rule that reproduces every labelled pair of the original task, training and test, yet differs from the intended rule on other inputs. At least 761 approved generators declare one or more as literal dictionaries (209 for ARC-AGI-2 training tasks, 549 for bank puzzles); the looser text count is 1,223. Each declared shortcut must be checked against every original pair and paired with generated inputs on which it fails. The fit check entered the semantic gate on 23 October; Section 5.3 shows approved generators that copy their whole rival list into the declaration and fail it.

Some generators record the strongest shortcut in their notes with its counterexample. Four of them:

- `dd038eb3d459`: "use the correct mapping for TR/BR/BL but rotate the green shape 180° for TL. It fits all five available canonical pairs because their only TL green object is a vertical three-cell bar and therefore 180°-symmetric."
- `be6269bef667`: "direction 3 continues through color 5 while other directions stop" fits all five original pairs; the sampler now emits a refuting witness in every draw.
- `60a8a9d74a75`: "fill only enclosed 8-walled regions touched by exactly one colored cell" fits all five original pairs; on the retained counterexample the intended rule changes 151 cells while the rival changes none.
- `aa0b08801e61`: "memorize the four masks in the public task and map any unknown mask to color 4" fits all four original pairs; the generator emits novel masks of the same symmetry class.

F Reviewer instructions (excerpts)

The October reviewers worked from a packet: a self-contained folder with the assigned generators, a bundled Racket runtime, tools and written instructions. The packet is not released. The excerpts below are from the last packet, of 26 October; earlier October packets assigned two or three targets

and differed in detail. We quote them because they record what reviewers were asked to treat as a meaningful rule and a useful lesson, which the rest of the paper can only summarize. Omissions are marked [. . .]; bracketed glosses are ours.

Task statement (`START-HERE.md`).

> The current trial is one full parametric build with deep adversarial self-review. [. . .] Spend the saved second-build effort trying to break your own artifact: attack shortcut rules and partial intermediates, [. . .] probe the real minimum lesson and relevant geometry/count/tie boundaries, and challenge authored certificates [the authored per-lesson check that exactly one declared setting fits the demonstrations] and public/private export. Repair findings, retain minimal regressions and refresh evidence on final bytes. [. . .] Finite checks do not prove universal uniqueness; keep remaining defects honest. [. . .] No training admission follows from a reviewer-ready return.

Lessons and rivals (`ARC-DESIGN.md`).

> A single-operation task is legitimate. Do not add operations merely to earn a category. [. . .] For each applicable axis [one of four reasoning demands: interacting rules, sequential state, contextual gating, task-defined symbols], identify an actual lesson or canonical pair [a pair from the original task] and a concrete simpler rival. Say what observation distinguishes them. A witness [an input on which the rival and the intended rule disagree] must change the relevant prediction: a cue that is present but redundant is weak teaching evidence. [. . .] finite declared-rival checks are not completeness. [. . .] Do not claim corpus coverage, model mastery or training eligibility from one repair annotation.

On rules with an intermediate step, using the guide's example of a ray traced from an emitter to the first object it hits:

> Make the dependency observable: retain a lawful witness where changing which object the ray hits changes the later action. Compare with a concrete rival that skips the hit calculation, such as always selecting the same color or acting from the emitter alone. Demonstrations should expose the difference. Merely naming a variable or writing several operations does not establish that the dependency matters.

What a parameter may change (`PARAMETRIC-RULES.md`).

> Preserve the original reasoning structure: object roles, spatial relationships, transformations and composition. Prefer a parameter that exposes a nearby meaningful behavior over a cosmetic reskin or a new unrelated puzzle. [. . .] Examples include a target or output color, direction, small count, or threshold. The parameter must change the executed transformation. Changing only input sizes, object positions, distractors, or sampling weights does not satisfy this request unless the actual rule also changes. [. . .] Choose parameter values once per episode from its deterministic seed, then use the same values for every demonstration and query in that episode. [. . .] Never resample the hidden rule independently for each pair. Demonstrations must distinguish the chosen behavior from plausible alternative parameter settings before a query requires that distinction.
>
> [. . .] finite demonstrations cannot prove uniqueness against every imaginable program. Keep difficult composition and generalization instead of padding every lesson to defeat arbitrary, contrived rules. [. . .] Inconsistent labels, low diversity, uncertainty about a rule, and two supported interpretations are different findings. A reviewer failing to solve a task is not evidence of ambiguity.

Ideas for rule diversity (`PARAMETRIC-EXAMPLES.md`).

> **Accepted:** `f47990415137` selects the component with most occupied cells, binds its bounding-box aspect ratio, then classifies other components relative to that ratio. [...] [From the proposed extensions that follow:] Teach occupied-area versus bbox-area selection with a sparse wide shape and a dense smaller box. Teach thresholds with below/equal/above-anchor sizes and changing anchors, defeating fixed-threshold shortcuts.
>
> "Ignore" can mean ineligible for selection, transparent to traversal, or protected from writes; these are different rules.
>
> Use 2–6 demos to distinguish plausible choices, then four private queries exposed one at a time: transfer, changed geometry/interaction, a relevant boundary, and substantive 30×30 content where lawful [allowed by the generator's input rules]. Prefer matched inputs across settings; incidental layout or palette should not reveal the rule. Avoid a fixed size-to-query-position mapping; an independent seed stream can shuffle query roles/sizes while preserving count-independent prefixes. Fit rivals to demos before inspecting query disagreement. A finite rival bank or native acceptance [the owner's acceptance of the family] never proves unrestricted uniqueness or learning benefit.